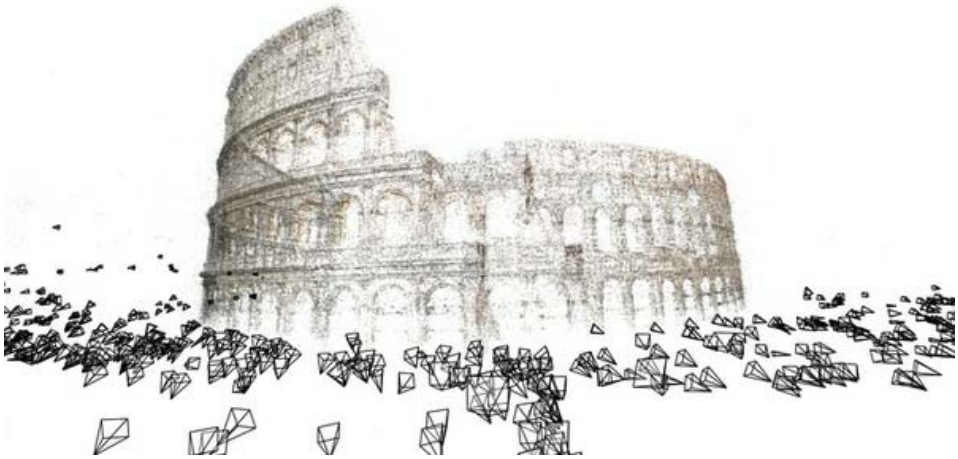
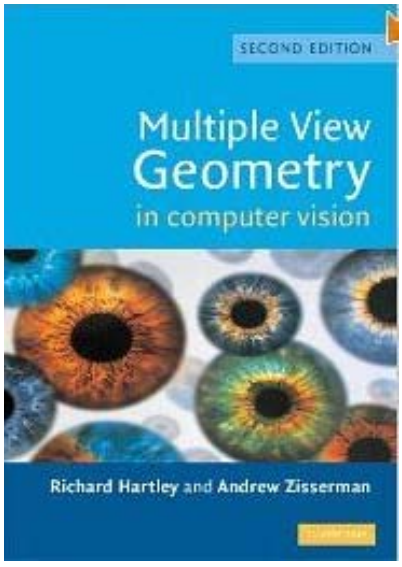
Structure from motion

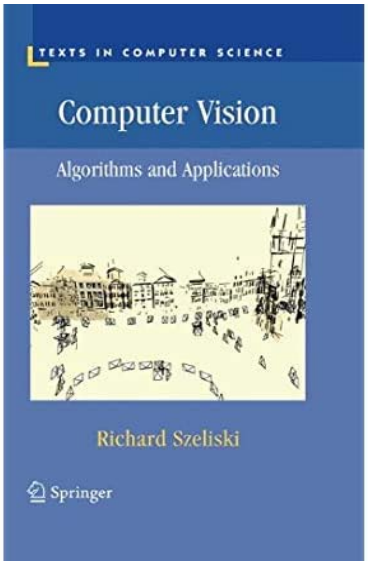


Slide credits: Ioannis Gkioulekas, Kris Kitani, Noah Snavely, Rob Fergus

Recommended Books



Only about geometry, very detailed



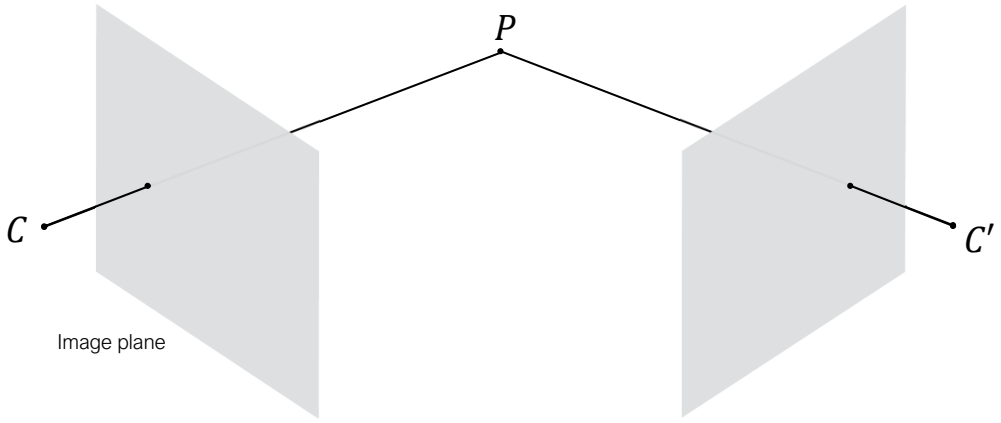
General computer vision, short geometry chapter

Overview of today's lecture

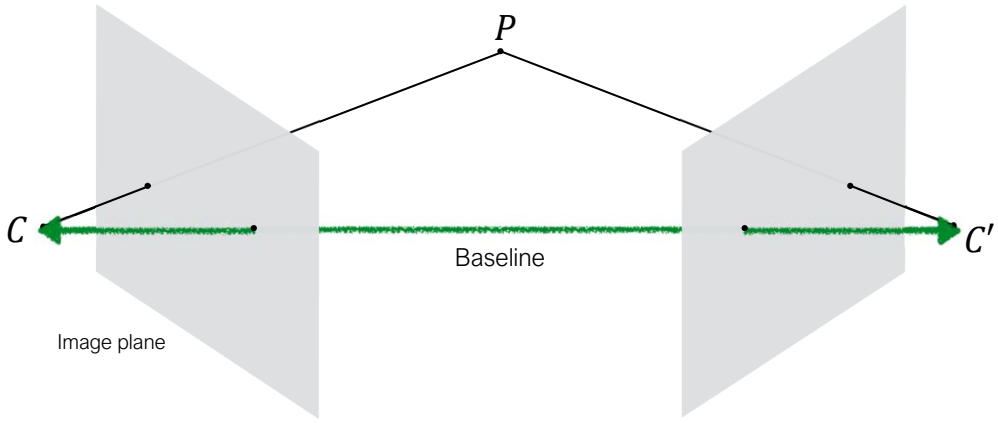
- Epipolar geometry review
- Estimating the F matrix
- A note on normalization.
- Two-view structure from motion.
- Ambiguities in structure from motion.
- Multi-view structure from motion.
- Large-scale structure from motion.
- Stereo

Epipolar geometry

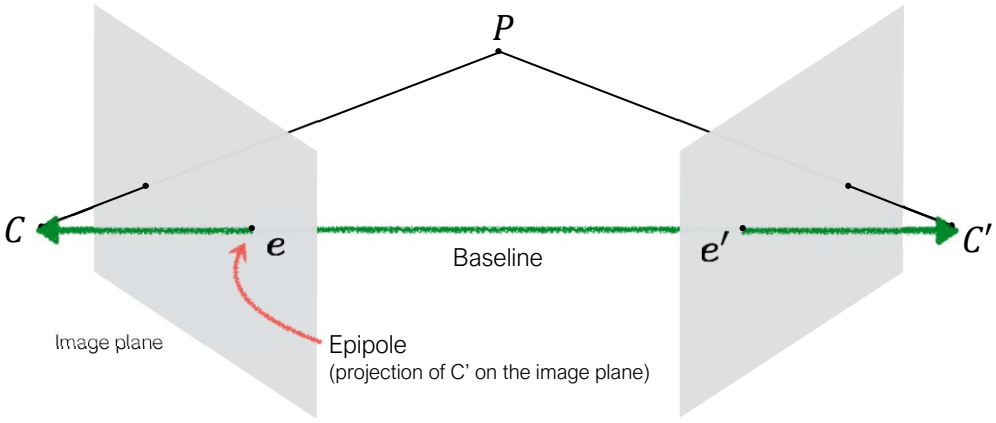
Epipolar geometry



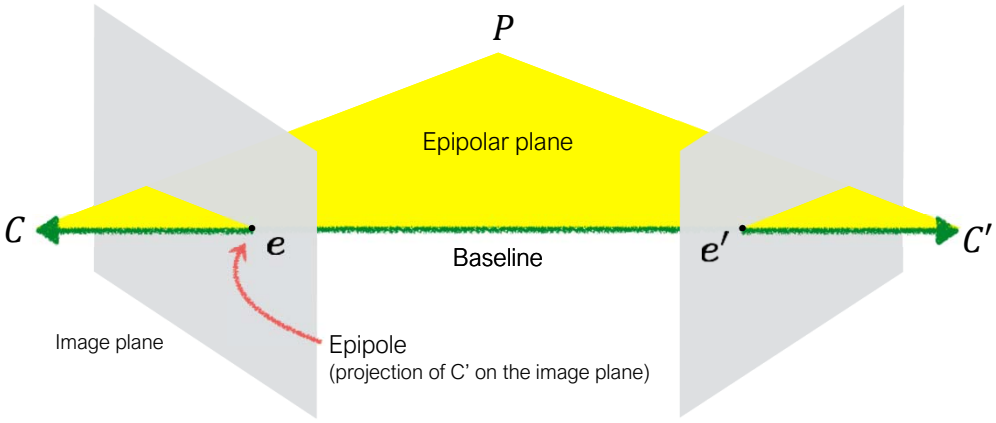
Epipolar geometry



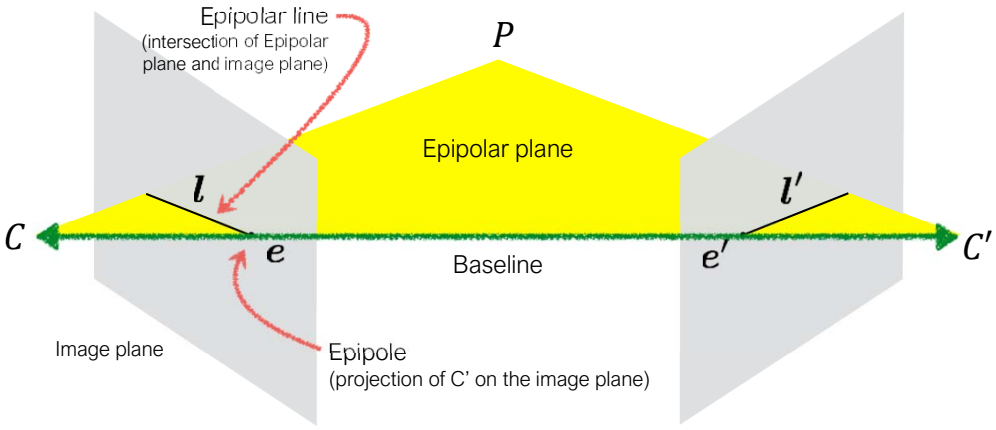
Epipolar geometry



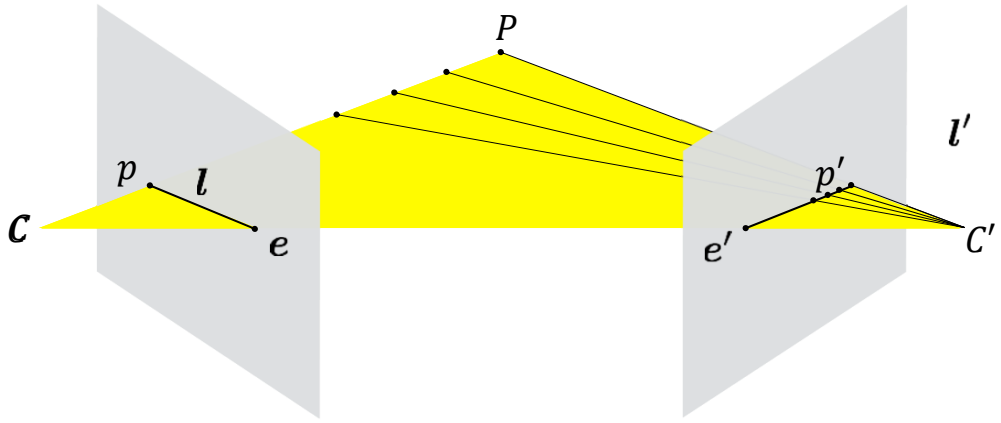
Epipolar geometry



Epipolar geometry

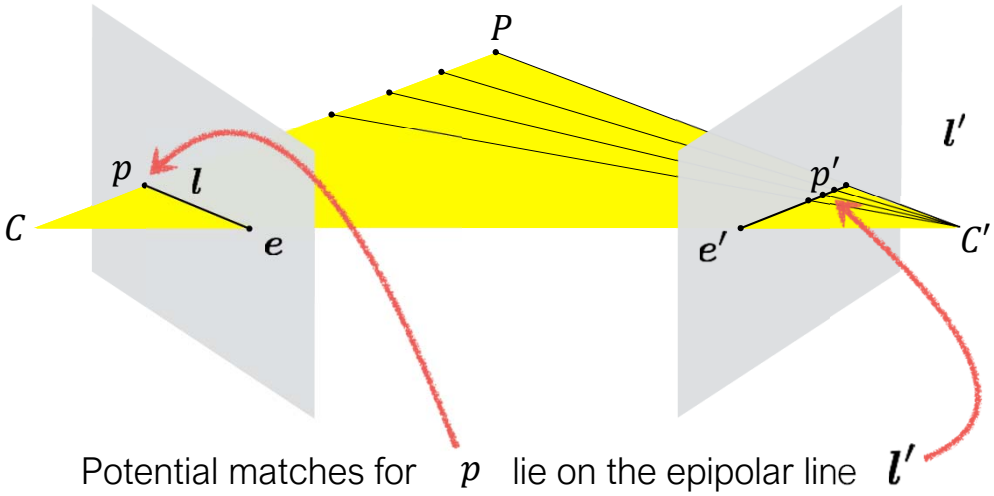


Epipolar constraint



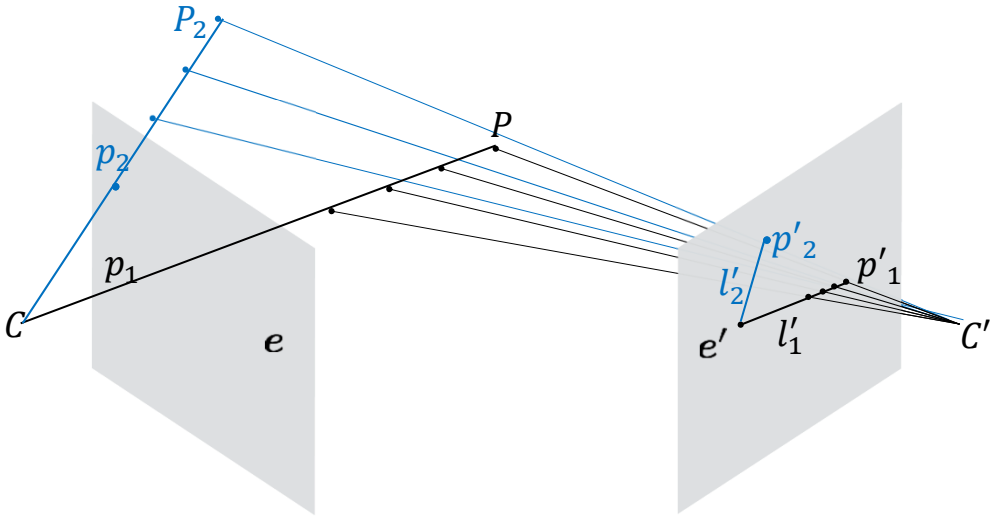
Potential matches for $\boldsymbol{p}$ lie on the epipolar line $\boldsymbol{l'}$

Epipolar constraint



Potential matches for p lie on the epipolar line l'

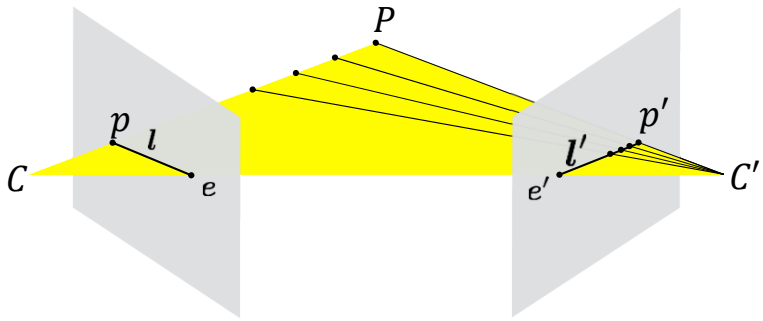
Multiple epipolar lines



Different points p, p_2 in image 1 define different rays of potential 3D points that could project to them. Hence they also define different epipolar lines l'_1, l'_2.
These lines intersect at the epipole e', since all 3D rays intersect at C and e' is the projection of C to camera 2

For different points p on camera 1 we get different rays of 3D points that could be mapped to them.
Different rays define different epipolar lines.

For every p, the ray of 3D points it defines is passing through the camera center C,
Since e' is the projection of C on camera 2, e' is also a point on the epipolar line l'.
So all the epipolar lines intersect at the epipole.



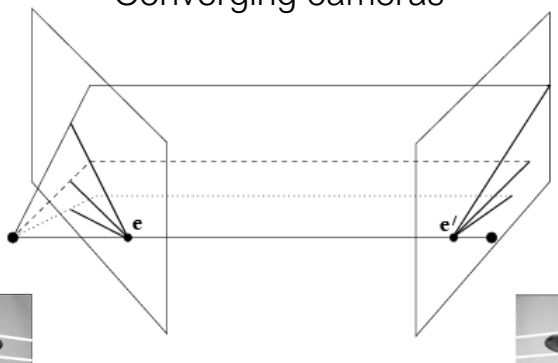
The point **p** (left image) maps to a Epipolar line in the right image

The baseline connects the Camera centers and epipols

An epipole **e** is a projection of the Camera center C' on the image plane

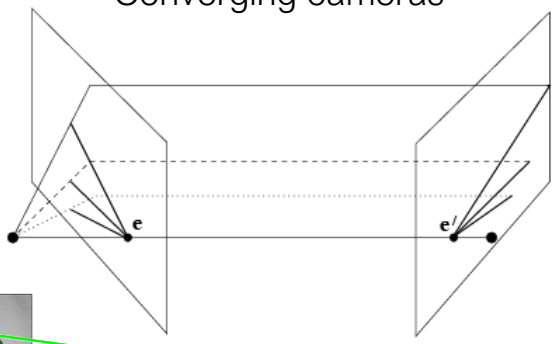
All epipolar lines in an image intersect at the epipole

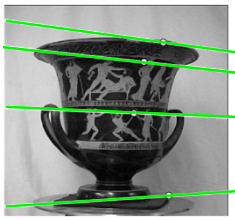
Converging cameras



Where is the epipole in this image?

Converging cameras





here!

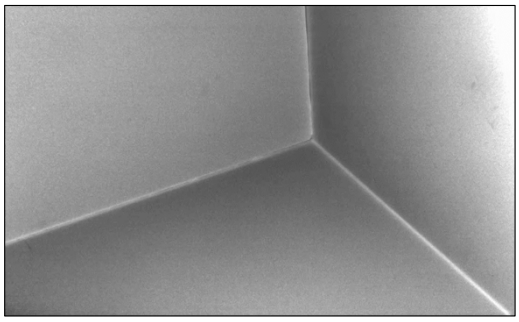
Where is the epipole in this image?

It's not always in the image

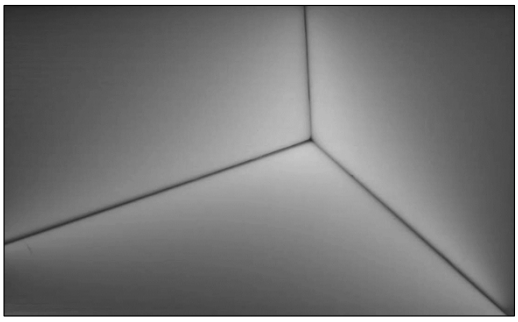
Plug: epipolar imaging.

A couple of interesting videos

Notice anything unusual about them?



Video stream 1

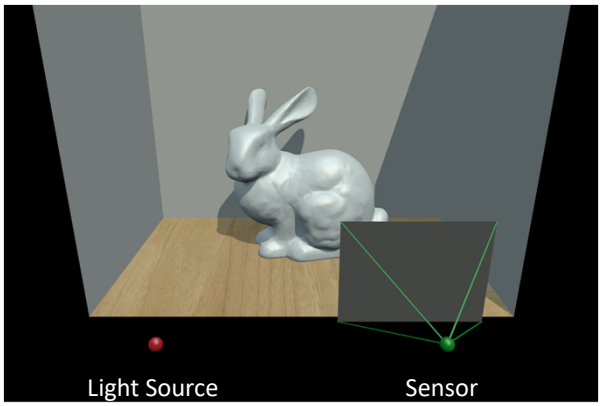


Video stream 2

Epipolar imaging camera

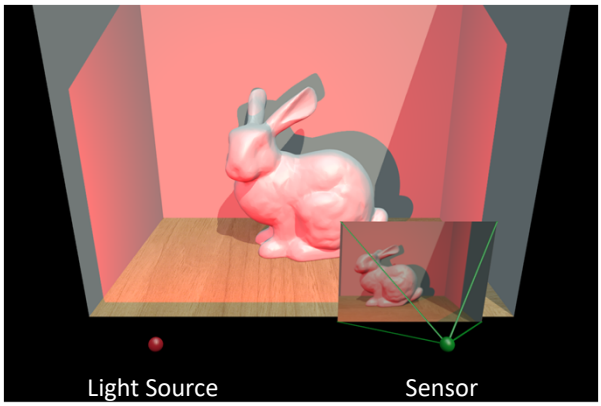
Built at CMU.

Regular Imaging



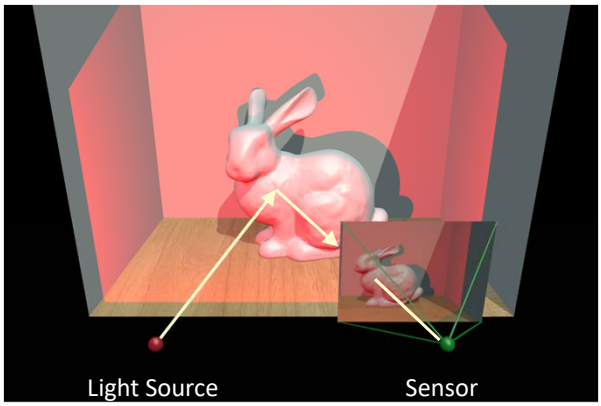
If you take apart any active illumination-based system, on the inside you'll find a light source and a sensor that images light reflected back from the scene.

Regular Imaging



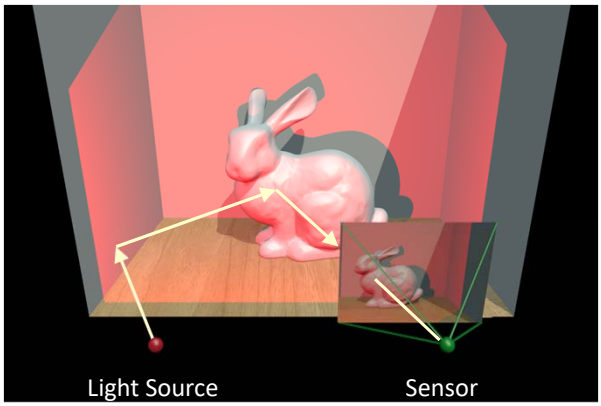
Today's CW ToF depth cameras operate in what we call 'regular imaging mode' - they illuminate and image their sensor's entire field-of-view at once. Because the output from the light source is spread over a large area, the sensor needs a long exposure to integrate enough reflected light.

Regular Imaging



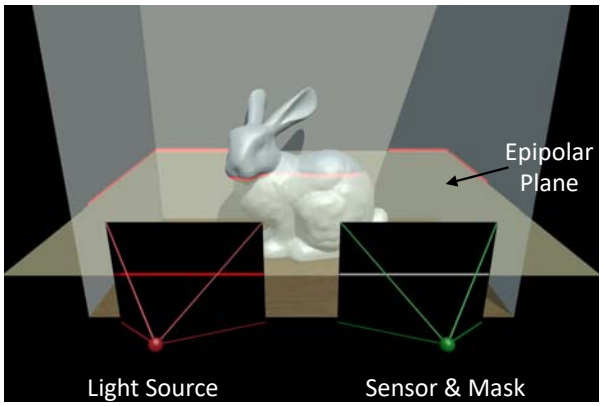
With regular imaging, the sensor integrates both direct, single-bounce light…

Regular Imaging



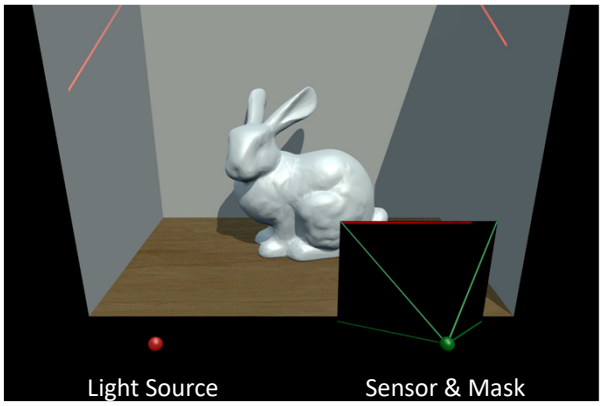
as well as light that takes indirect, multi-bounce paths from the source to the sensor.

Epipolar Imaging



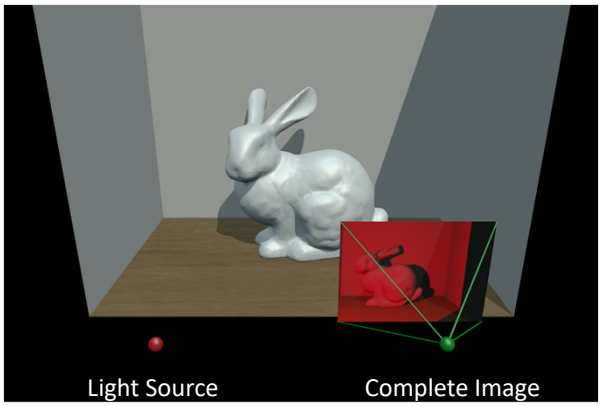
In an epipolar imaging system, the light source and sensor are placed in a rectified stereo configuration so that a horizontal stripe of light from the source corresponds to a horizontal row of pixels on the sensor. A configurable mask placed in front of the sensor is set to expose only that single row.

Epipolar Imaging



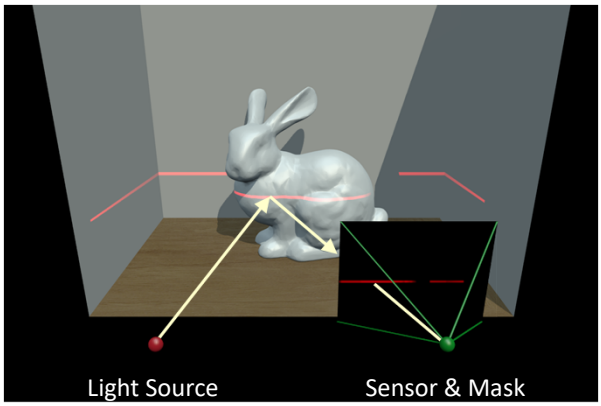
To form an image, the light stripe is swept across the scene while the exposed row on the sensor moves in lock-step. Because the source's output is concentrated into a single line, each row needs to be exposed only for a very short duration to integrate a useable signal.

Epipolar Imaging



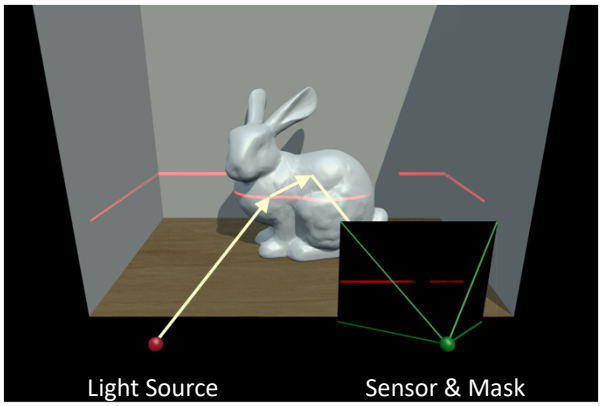
At the end of the sweep we have a complete epipolar image on the sensor. In a line striping system, we would need to read out an entire image for each projected stripe which would be very slow. In epipolar imaging, each pixel is read out only once.

Epipolar Imaging



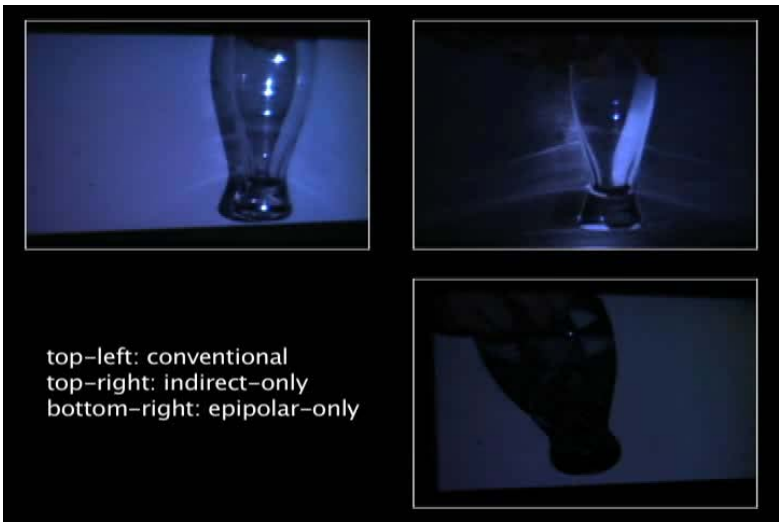
With epipolar imaging, the sensor only collects light paths that follow the epipolar constraint between the source and sensor. This allows direct, single-bounce paths through.

Epipolar Imaging



But blocks almost all multi-bounce paths from reaching the sensor because they don't follow epipolar geometry.


top-left: conventional
top-right: indirect-only
bottom-right: epipolar-only


top-left: conventional
top-right: indirect-only
bottom-right: epipolar-only


top-left: conventional
top-right: indirect-only
bottom-right: epipolar-only

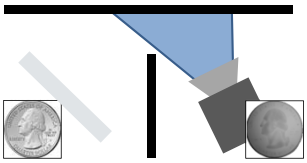

top-left: conventional
top-right: indirect-only
bottom-right: epipolar-only

Computational Photography

Learn about this and other unconventional cameras
(Come to my class in the spring semester)

cameras that take video at the speed of light

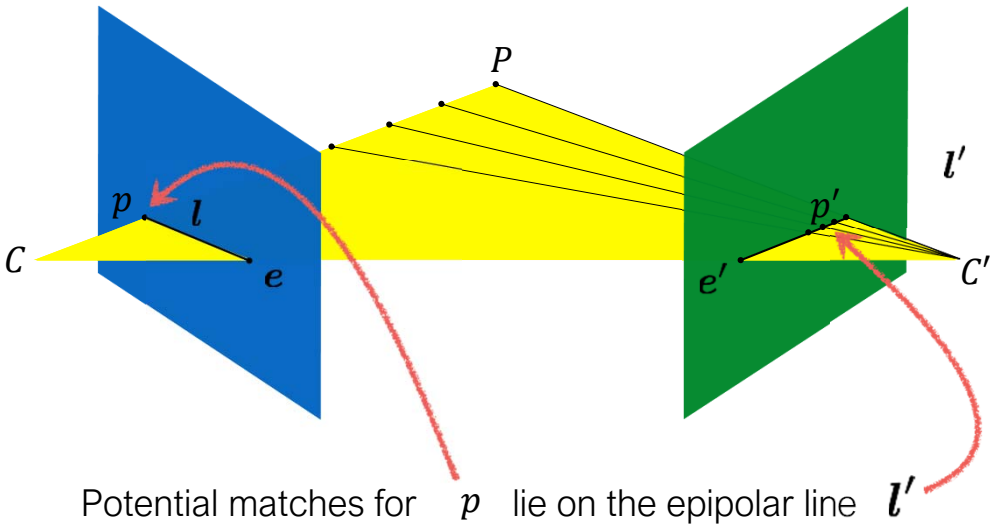
cameras that measure depth in real time



cameras that see around corners

cameras that capture entire focal stacks

Recall:Epipolar constraint

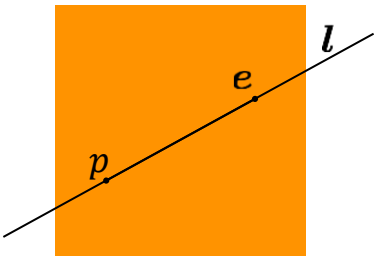


Potential matches for p lie on the epipolar line l'

Representing the …

Epipolar Line

$$ax + by + c = 0$$ in vector form $$\boldsymbol{l} = \begin{bmatrix} a \\ b \\ c \end{bmatrix}$$

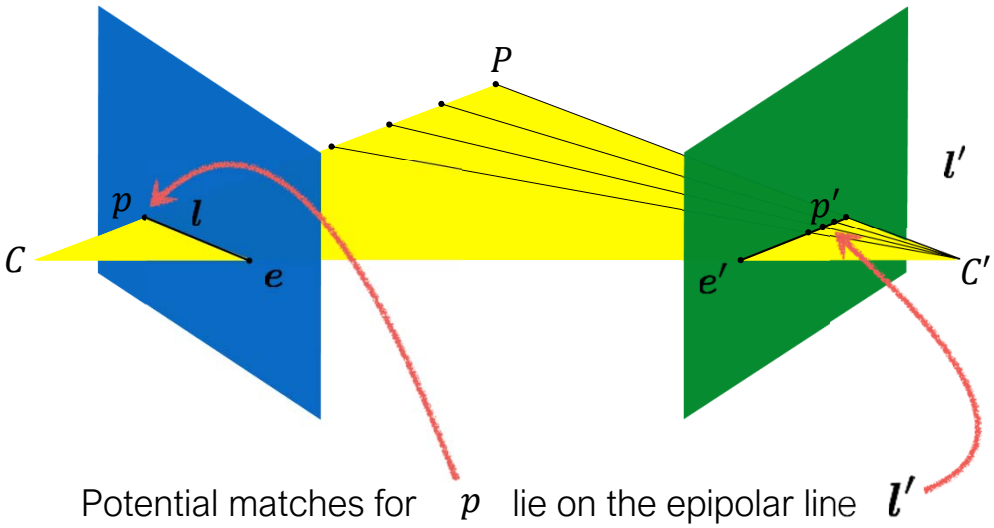


If the point p is on the epipolar line $\boldsymbol{l}$ then

$$p^T l = 0$$

The essential matrix

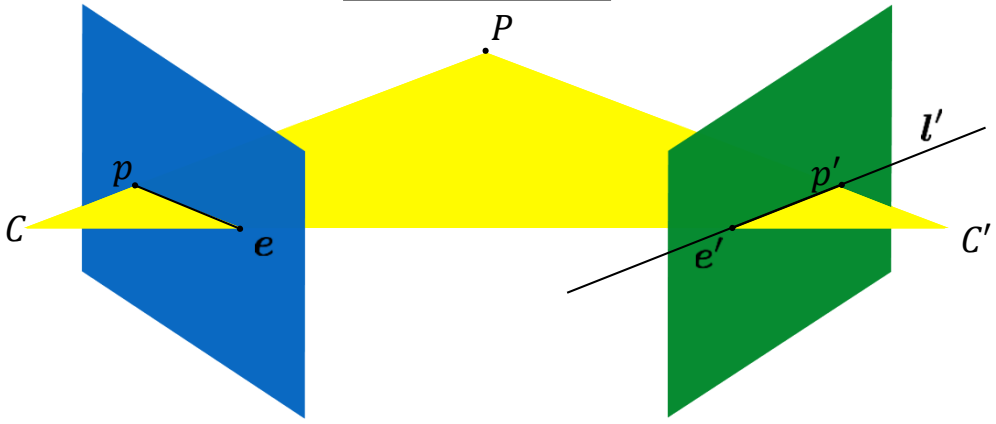
Recall:Epipolar constraint



Potential matches for p lie on the epipolar line l'

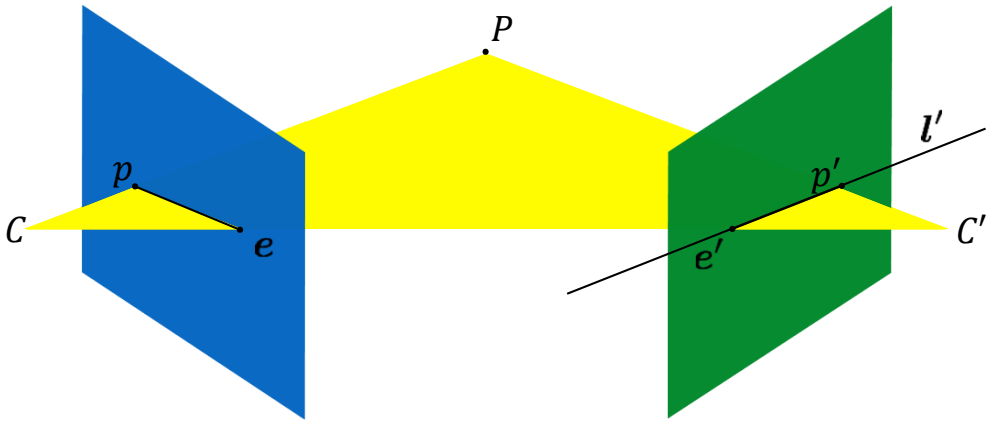
Given a point in one image,
multiplying by the **essential matrix** will tell us
the **epipolar line** in the second view.

$$Ep = l'$$

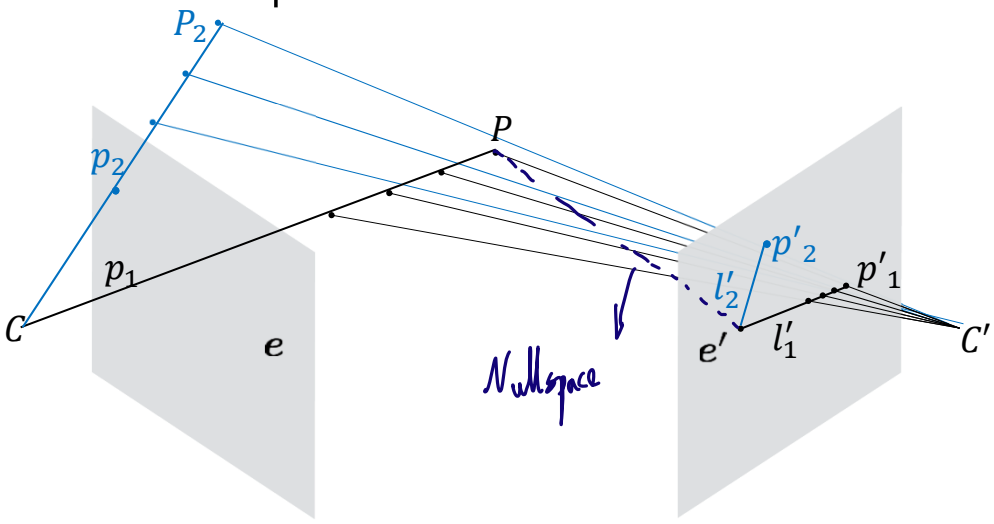


So if $p'^T l' = 0$ and $Ep = l'$ then

$$p' E p = 0$$



The nullspace of the essential matrix



for every p: $e'^T l' = 0 \Longrightarrow e'^T E p = 0 \Longrightarrow e'^T E = 0$)

We also so that all epipolar lines intersect at the epipole.

From that we can conclude that e'^TE=0 Ee=0

properties of the E matrix

Longuet-Higgins equation	$p'^T E p = 0$	
Epipolar lines	$p^T l = 0$ $l' = Ep$	$p'^T l' = 0$ $l = E^T p'$
Epipoles	$e'^T E = 0$	$Ee = 0$

Let's now summarize the properties of the essential matrix. Remember that, every time we are working with the essential matrix, we are expressing 2D points in homogeneous coordinates in the *camera* coordinate system, ignoring camera intrinsics. We have first the Longuet-Higgins equation, which says that left-and-right multiplying the essential matrix by two image points satisfying the epipolar constraint gives us zero.

Second, we have the property we started from: Given a point on one image, we can find the corresponding epipolar line on the other image through multiplication by the essential matrix.

Lastly, we have that when multiplying the epipoles with the essential matrix, we get zero. This mathematically expresses the fact that all epipolar lines pass through the two epipoles.

In all of this, we have been expressing image points in camera coordinates, which in practice only works if our cameras have intrinsic matrices that are

identities. In the next slides, we will see how to remove this assumption.

Recall: camera matrices

We can write everything into a single projection:

$$\mathbf{p} = \boldsymbol{M}\boldsymbol{P}_{\mathbf{w}}$$

The camera matrix now looks like:

$$\mathbf{M} = \mathbf{K}[\mathbf{R}|-\mathbf{R}\mathbf{C}]$$

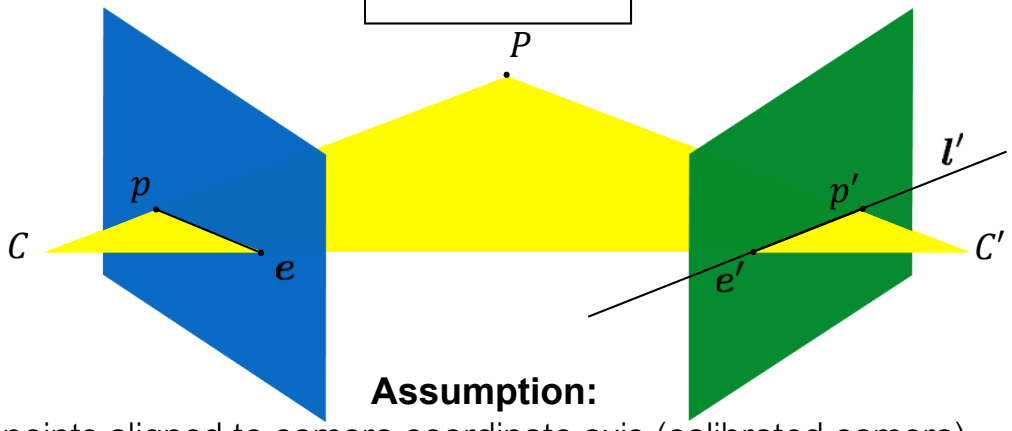
$$\mathbf{M} = \begin{bmatrix} f & 0 & p_x \\ 0 & f & p_y \\ 0 & 0 & 1 \end{bmatrix} \left[\mathbf{R} \;\middle|\; -\mathbf{R}\mathbf{C}\right]$$

intrinsic parameters (3 x 3): correspond to camera internals (sensor not at f = 1 and origin shift)

extrinsic parameters (3 x 4): correspond to camera externals (world-to-image transformation)

Given a point in one image,
multiplying by the **essential matrix** will tell us
the **epipolar line** in the second view.

$$l' = E \quad p$$



Assumption:
points aligned to camera coordinate axis (calibrated camera)
(internal matrix K was pre-applied)

The
Fundamental matrix
is a
generalization
of the
Essential matrix,
where the assumption of
calibrated cameras
is removed

we will see a different matrix that we will call the fundamental matrix. This is a generalization of the essential matrix that allows us to remove the assumption of identity intrinsic matrices, and express all image points on the image coordinate system.

$$\widehat{p'}^{T} E \hat{p} = 0$$

The Essential matrix operates on image points expressed in
normalized coordinates
(points have been aligned (normalized) to camera coordinates)

$$\widehat{p'} = K^{-1} p' \qquad \underset{\text{camera point}}{\hat{p}} = K^{-1} \underset{\text{image point}}{p}$$

We have how the essential matrix and the Longuet-Higgins equation can be used to check whether two image points, expressed in the camera coordinate system, satisfy the epipolar constraint. As we saw in the Camera Models lecture, for each of the two points, we can relate image coordinates and camera coordinates using the corresponding camera intrinsic matrices K and K'.

$$\hat{p}'^{T} E \hat{p} = 0$$

The Essential matrix operates on image points expressed in
normalized coordinates
(points have been aligned (normalized) to camera coordinates)

$$\hat{p}' = K'^{-1} p' \qquad \hat{p} = K^{-1} p$$

$\hat{p}$: camera point; p: image point

Writing out the epipolar constraint in terms of image coordinates

$$p'^{T} K'^{-T} E K^{-1} p = 0$$

$$p'^{T} (K'^{-T} E K^{-1}) p = 0$$

$$p'^{T} F p = 0$$

We can then replace the camera coordinates with image coordinates in the Longuet-Higgins equation, and define a new matrix F that combines the essential matrix and the intrinsic matrices.

Same equation works in image coordinates!

$$p'^T F p = 0$$

it maps pixels to epipolar lines

With this simple procedure, we arrive at a new equation that is identical to the Longuet-Higgins equation, but is now expressed in image coordinates. We call this new matrix F the fundamental matrix.

properties of the E matrix

Longuet-Higgins equation	$p'^T E p = 0$	
Epipolar lines	$p^T l = 0$	$p'^T l' = 0$
	$l' = E\ p$	$l = E^T p'$
Epipoles	$e'^T E = 0$	$E\ e = 0$

The fundamental matrix has all the same properties as the essential matrix, except now with 2D points expressed in the image coordinate system. These include the Longuet-Higgins equation, the mapping from image points to epipolar lines, and the definition of the epipoles.

Breaking down the fundamental matrix

$$\mathbf{F} = \mathbf{K}'^{-\top}\mathbf{E}\mathbf{K}^{-1}$$

$$\mathbf{F} = \mathbf{K}'^{-\top}[\mathbf{t}_\times]\mathbf{R}\mathbf{K}^{-1}$$

Depends on both intrinsic and extrinsic parameters

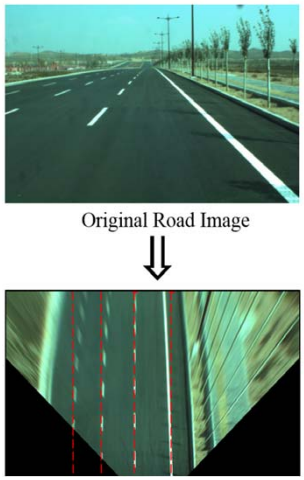
We had previously expressed the essential matrix in terms of the camera extrinscis, namely the rotation vector R and translation vector t relating their coordinate systems. Given how we defined the fundamental matrix, we can use those, along with the camera intrinsic matrices, to express the fundamental matrix. This expression says that, if we have a fully calibrated pair of cameras, we can use their intrinsic and extrinsic matrices to compute their fundamental matrix. We will see in the next slides that calibrating the two cameras is not necessary for computing their fundamental matrix. Instead, we can compute it using a set of matched point pairs between two images captured by the two cameras.

Differences between a fundamental matrix and an homography

Homography: maps point to point
Fundemental matrix only maps points to lines

Homography applies only to planes
Fundemental matrix explains the entire 3D scene

Caution: If your scene is planer or the cameras have a join center, it can be explained by an homography but this homography is *not* the Fundemental matrix



Breaking down the fundamental matrix

$$\mathbf{F} = \mathbf{K}'^{-\top}\mathbf{E}\mathbf{K}^{-1}$$

$$\mathbf{F} = \mathbf{K}'^{-\top}[\mathbf{t}_\times]\mathbf{R}\mathbf{K}^{-1}$$

Depends on both intrinsic and extrinsic parameters

How would you solve for F?

$$p_m'^T F p_m = 0$$

The 8-point algorithm

Assume you have ***M*** matched *image* points

$$\{ p_j, p'_j \}$$

Each correspondence should satisfy

$$p'^T_j F p_j = 0$$

How would you solve for the 3 x 3 ***F*** *matrix?*

Set up a homogeneous linear system with 9 unknowns

S V D

In the previous slides, we saw how we can compute the fundamental matrix using the intrinsics and extrinsics of two calibrated cameras. In this submodule, we will see how to do the same using a set of matched image points. In particular, we assume we have a set of N pairs of points on the left and right images that are matched, meaning that they look at the same 3D point in the scene. Given this, each image point pair satisfies the Longuet-Higgins equation. Our task will be to solve for the 3x3 fundamental matrix F. We will use the same high-level procedure we have been using for all these reconstruction tasks: we will try to express our constraints in the form of a linear system that we can then solve for the unknown fundamental matrix F.

$$p_m'^T F p_m = 0$$

$$\begin{bmatrix} x'_m & y'_m & 1 \end{bmatrix} \begin{bmatrix} f_1 & f_2 & f_3 \\ f_4 & f_5 & f_6 \\ f_7 & f_8 & f_9 \end{bmatrix} \begin{bmatrix} x_m \\ y_m \\ 1 \end{bmatrix} = 0$$

How many equation do you get from one correspondence?

$$\begin{bmatrix} x'_m & y'_m & 1 \end{bmatrix} \begin{bmatrix} f_1 & f_2 & f_3 \\ f_4 & f_5 & f_6 \\ f_7 & f_8 & f_9 \end{bmatrix} \begin{bmatrix} x_m \\ y_m \\ 1 \end{bmatrix} = 0$$

Set up a homogeneous linear system with 9 unknowns

$$\begin{bmatrix} x_1x'_1 & x_1y'_1 & x_1 & y_1x'_1 & y_1y'_1 & y_1 & x'_1 & y'_1 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ x_Mx'_M & x_My'_M & x_M & y_Mx'_M & y_My'_M & y_M & x'_M & y'_M & 1 \end{bmatrix} \begin{bmatrix} f_1 \\ f_2 \\ f_3 \\ f_4 \\ f_5 \\ f_6 \\ f_7 \\ f_8 \\ f_9 \end{bmatrix} = \mathbf{0}$$

How many equations do you need?

We start by writing out the Longuet-Higgins equation for one image pair.

we can expand out the matrix vector multiplications to express this as a long linear equation involving point coordinates (entries of the p,p' points) and matrix entries.
We reshape the 3x3 elements of F into an unknown 9x1 vector.

Stacking equations from all point pairs gives us a homogeneous linear system, where the unknowns are the 9 entries of the 3x3 fundamental matrix.

As always, we can solve this homogeneous linear system using the SVD.

Looking at the system, we see that we have 8 degrees of freedom. The system has 9 unknowns, but remember that fundamental matrices use

homogeneous coordinates, and therefore they are defined up to scale, which removes one degree of freedom.

Each point pair (according to epipolar constraint)
contributes only one scalar equation

$$p_m'^T F p_m = 0$$

Note: This is different from the Homography estimation where each point pair contributes 2 equations.

We need at least 8 points

Hence, the 8 point algorithm!

We also see that every point pair provides us with one constraint on the unknowns.

Therefore, we need at least 8 point pairs to uniquely determine the fundamental matrix, which is where the name 8-point algorithm comes from.

How do you solve a homogeneous linear system?

$$\mathbf{A}\boldsymbol{X} = \mathbf{0}$$

Total Least Squares

minimize $\|\mathbf{A}\boldsymbol{x}\|^2$

subject to $\|\boldsymbol{x}\|^2 = 1$

S V D !

Remember also, from previous lectures, that when we solve homogeneous linear systems, we add a unit-norm constraint. That constraint corresponds to the one degree of freedom we lose due to scale invariance.

As before we solve this with SVD.

Eight-Point Algorithm

0. (Normalize points)
1. Construct the M x 9 matrix **A**
2. Find the SVD of **A**
3. Entries of **F** are the elements of column of **V** corresponding to the least singular value
4. (Enforce rank 2 constraint on F)
5. (Un-normalize F)

Here is a summary of the eight point algorithm. Note that, in green, there are a few steps that we have not described. We will discuss the normalization and unnormalization steps in the next slides.

Eight-Point Algorithm

0. (Normalize points)
1. Construct the M x 9 matrix **A**
2. Find the SVD of **A**
3. Entries of **F** are the elements of column of **V** corresponding to the least singular value
4. (Enforce rank 2 constraint on F)
5. (Un-normalize F)

Here F is a 9x1 vector

This is a 2nd SVD on F itself, now rearranged as a 3x3 matrix

The fundamental matrix should be a rank 2 matrix. This results from the fact that Fe=0.
However when we solved for F we expressed it as a vector of 9x1 numbers. When we solve this system we get a 9 elements vector out, and we reshape them into a 3x3 matrix. This matrix is unlikely to be a rank 2 matrix because it is a non linear constraint on the elements of F, that we did not expressed in the formation of the constraints in A.

As we described in the Homographies lecture, we can enforce the rank 2 constraint by computing the singular value decomposition of F, and setting its smallest singular value to zero.

Note that we use here SVD twice. First SVD on the big matrix A to get the 9 elements of F.
We then reshape them into a 3x3 matrix and apply SVD on this 3x3 matrix to make it a rank 2 matrix

Reminder from lecture 6: Using SVD to enforce rank constraints

Problem statement: Given

- an $M \times N$ matrix F,
- an integer $k < \min(M, N)$,

find the matrix F' of rank k that is closest to F in the least-squares sense

$$\min_{\substack{F' \\ \text{rank}(F')=k}} \|F - F'\|^2$$

Solution:

- compute the SVD $F = U \cdot \Sigma \cdot V^\top$
- form a diagonal matrix Σ' by replacing all but the k smallest singular values in Σ with 0.
- Compute the solution as $F' = U \cdot \Sigma' \cdot V^\top$

Before we finish this linear algebra detour, I want to mention one more property of SVD we will be using in later lectures. Consider the following problem: We have an M by N matrix F, and we want to approximate it, in the least squares sense, with a new matrix F', of some lower rank k.

We can use the SVD of F to compute the new matrix F' as follows: Take the singular value matrix Sigma, set all singular values except for the k largest to zero, then form F' by left-and-right multiplying the resulting matrix with U and V-transpose.

Eight-Point Algorithm

Note: in practice F has only 7 degrees of freedom (why?),

0. (Normalize points)
1. Construct the M x 9 matrix **A**
2. Find the SVD of **A**
3. Entries of **F** are the elements of column of **V** corresponding to the least singular value
4. (Enforce rank 2 constraint on F)
5. (Un-normalize F)

Note that in practice this implies that the Fundemental matrix has only 7 degrees of freedom not 8.
The fact that F needs to be a rank 1 matrix adds an additional constraint.

However this is a non linear constraint and we do not use it as part of our linear system when forming the matrix A. Therefore to solve for F using a linear system we need at least 8 points.

If we have only 7 points we will need to include non linear constraints.

Eight-Point Algorithm

Note: in practice F has only 7 degrees of freedom (rank 2), but the last constraint is non linear and is not used by the 8 point algorithm

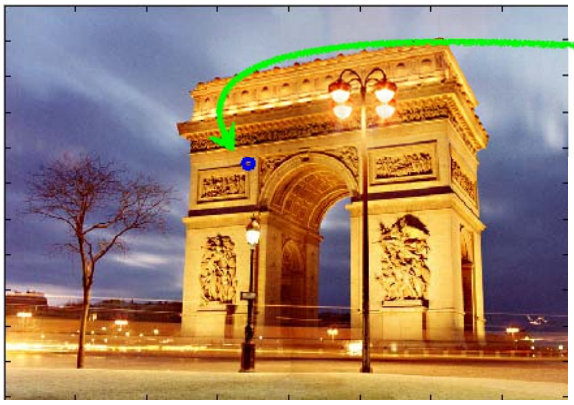
0. (Normalize points)
1. Construct the M x 9 matrix **A**
2. Find the SVD of **A**
3. Entries of **F** are the elements of column of **V** corresponding to the least singular value
4. (Enforce rank 2 constraint on F)
5. (Un-normalize F)


Let’s look at a quick example of how this all works out. We are given these two images of the same scene, the arc de triomphe in Paris. We do not have any information about the cameras that captured these two images, we just know that they are looking at the same scene.

Epipolar lines

We can use the techniques from the Feature Detection and Matching lecture to find at least 8 matching point pairs between the two images. We can then use those as input to our implementation of the 8-point algorithm, to compute the fundamental matrix relating the two images. Using the fundamental matrix, we can, for example, draw corresponding epipolar lines on the two images.

$$\mathbf{F} = \begin{bmatrix} -0.00310695 & -0.0025646 & 2.96584 \\ -0.028094 & -0.00771621 & 56.3813 \\ 13.1905 & -29.2007 & -9999.79 \end{bmatrix}$$



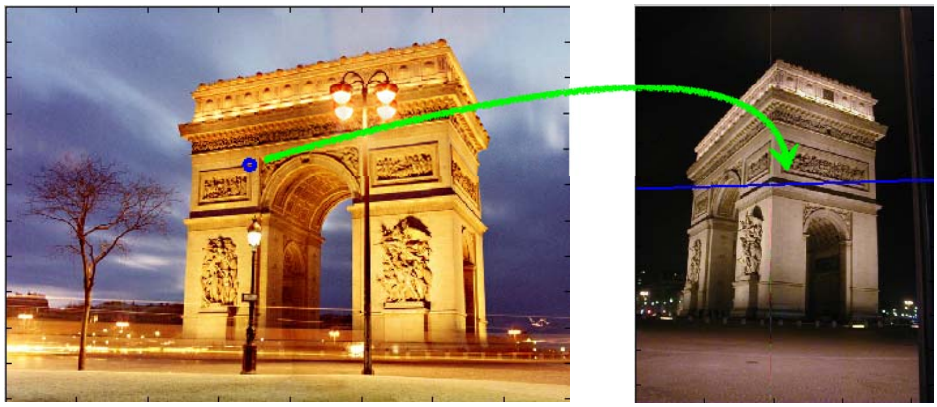
$$p = \begin{bmatrix} 343.53 \\ 221.70 \\ 1.0 \end{bmatrix}$$

$$l' = Fp = \begin{bmatrix} 0.0295 \\ 0.9996 \\ -265.1531 \end{bmatrix}$$

With the fundamental matrix at hand, we can also take some arbitrary point on image one…

… and compute and draw the corresponding epipolar line on image two. Note that the epipolar line indeed passes through the point corresponding to the corner we selected in image one.

$$l' = Fp$$
$$= \begin{bmatrix} 0.0295 \\ 0.9996 \\ -265.1531 \end{bmatrix}$$



With the fundamental matrix at hand, we can also take some arbitrary point on image one…

… and compute and draw the corresponding epipolar line on image two. Note that the epipolar line indeed passes through the point corresponding to the corner we selected in image one.

Where is the epipole?

How would you compute it?

Lastly, remember that the epipolar lines we drew intersect at the epipole. The epipole may lay outside the image boundaries.

$$\mathbf{F}\boldsymbol{e} = \mathbf{0}$$

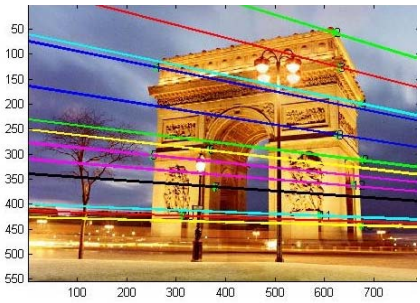
The epipole is in the right null space of **F**

How would you solve for the epipole?

(hint: this is a homogeneous linear system)

S V D !

As we saw earlier, the epipole when multiplied by the fundamental matrix F gives us zero. Therefore, we can compute the epipole by solving a homogeneous linear system involving the fundamental matrix.




```
>> [u,d] = eigs(F' * F)
```

eigenvectors

```
u =
    -0.0013    0.2586   -0.9660
     0.0029   -0.9660   -0.2586
     1.0000    0.0032   -0.0005
```

eigenvalue

```
d = 1.0e8*
   -1.0000         0         0
         0   -0.0000         0
         0         0   -0.0000
```

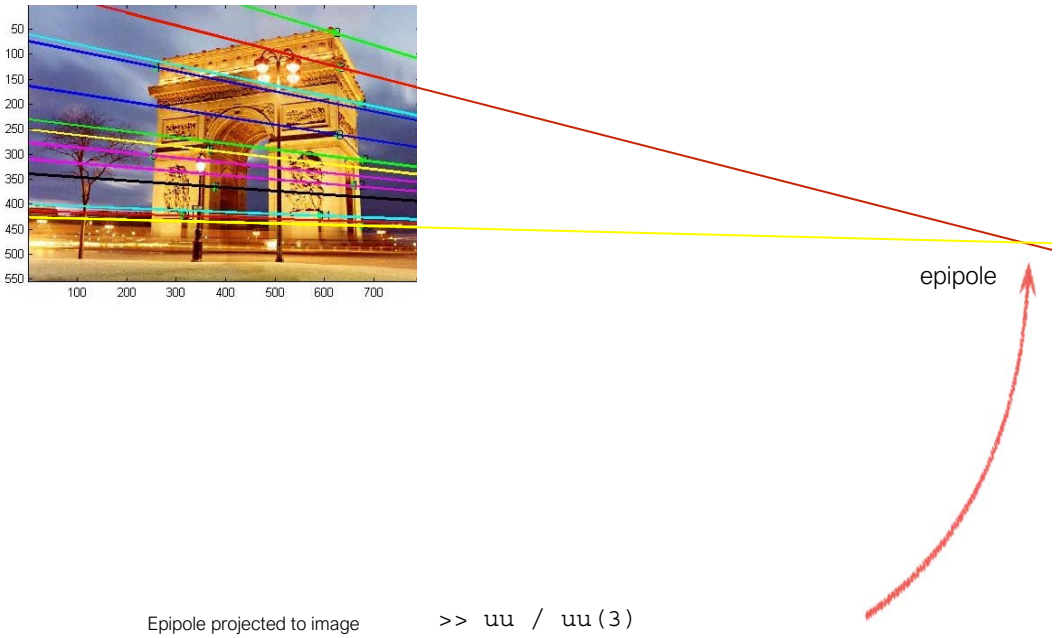
Eigenvector associated with smallest eigenvalue

```
>> uu = u(:,3)
(  -0.9660    -0.2586    -0.0005)
```

Epipole projected to image coordinates

```
>> uu / uu(3)
(1861.02     498.21    1.0)
```

this means that we can find the epipole by simply computing the SVD of F, and taking the right singular vector corresponding to the smallest singular value.



Epipole projected to image coordinates

```
>> uu / uu(3)
(1861.02      498.21     1.0)
```

The Fundamental Matrix Song

https://www.youtube.com/watch?v=DgGV3l82NTk

A note on normalization

Estimating F – 8-point algorithm

- The fundamental matrix F is defined by

$$p_m'^T F p_m = 0$$

for any pair of matches p and p' in two images.

- Let x=(*u,v,1*)^T and x'=(*u',v',1*)^T,

$$\mathbf{F} = \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix}$$

each match gives a linear equation

$$uu' f_{11} + vu' f_{12} + u' f_{13} + uv' f_{21} + vv' f_{22} + v' f_{23} + u f_{31} + v f_{32} + f_{33} = 0$$

We will now explain why we need to use normalization to make the 8-point algorithm work robustly. For this, we can take a look at the homogeneous linear system we are solving to compute the fundamental matrix F.

Problem with 8-point algorithm

$$\begin{bmatrix} u_1u_1' & v_1u_1' & u_1' & u_1v_1' & v_1v_1' & v_1' & u_1 & v_1 & 1 \\ u_2u_2' & v_2u_2' & u_2' & u_2v_2' & v_2v_2' & v_2' & u_2 & v_2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_nu_n' & v_nu_n' & u_n' & u_nv_n' & v_nv_n' & v_n' & u_n & v_n & 1 \end{bmatrix} \begin{bmatrix} f_{11} \\ f_{12} \\ f_{13} \\ f_{21} \\ f_{22} \\ f_{23} \\ f_{31} \\ f_{32} \\ f_{33} \end{bmatrix} = 0$$

~10000 ~10000 ~100 ~10000 ~10000 ~100 ~100 ~100 1

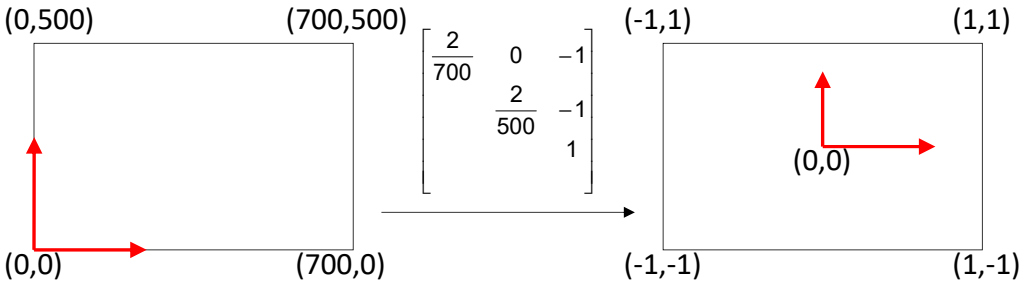
Orders of magnitude difference
between column of data matrix
→ least-squares yields poor results

Note that some of the columns involve products of two image coordinates, some involve individual image coordinates, and one column is equal to just ones. If our images have, say, 1024-by-1024 pixels, this means that the columns involving products of image coordinates will have entries magnitude around 10,000; and the columns involving individual image coordinates will have entries of magnitude around 100. We see that there are columns spanning four different orders of magnitude in the matrix of our linear system. This is problematic, because our marking of the points p,p' in the images will have noise. least-squares procedures are not robust in such situations: the results tend to overemphasize small errors in the large-magnitude columns, and completely ignore the small-magnitude ones.

Normalized 8-point algorithm

normalized least squares yields good results

Transform image to ~[-1,1]x[-1,1]



Fortunately, there is a very simple way to deal with this issue. Before running the 8-point algorithm, we can apply a similarity transform to each image, to map it to the [-1,1] by [-1 1] cube. As a consequence, the coordinates of the image points we use as input to the 8-point algorithm will now all have magnitudes between [0 1], rather than between [0 1000].

Normalized 8-point algorithm

1. Transform input by $\widehat{p_i} = Tp_i$, $\widehat{p_i'} = Tp'_i$
2. Call 8-point on $\widehat{p_i}$, $\widehat{p_i'}$ to obtain $\hat{F}$
3. $F = T'^T \hat{F}\, T$

$$p'^T F p = 0$$

$$\hat{p}^T T'^{-T} \; F \; T^{-1}\hat{p} = 0$$

$$\hat{F}$$

So, here is how this works. First, we transform the input points using the similarity transform we just described. Second, we run the 8-point algorithm on the transformed image-point pairs, to compute an intermediate fundamental matrix F. This intermediate matrix is not what we want, given that it applies to points in the transformed image plane, and not the original image plane. To deal with this, our last step is to transform the intermediate to the final fundamental matrix by using multiplications with the similarity transforms.

To see where the expression mapping the intermediate to the final fundamental matrix comes from, consider first the Longuet-Higgins equation between the final fundamental matrix and the unnormalized image points.

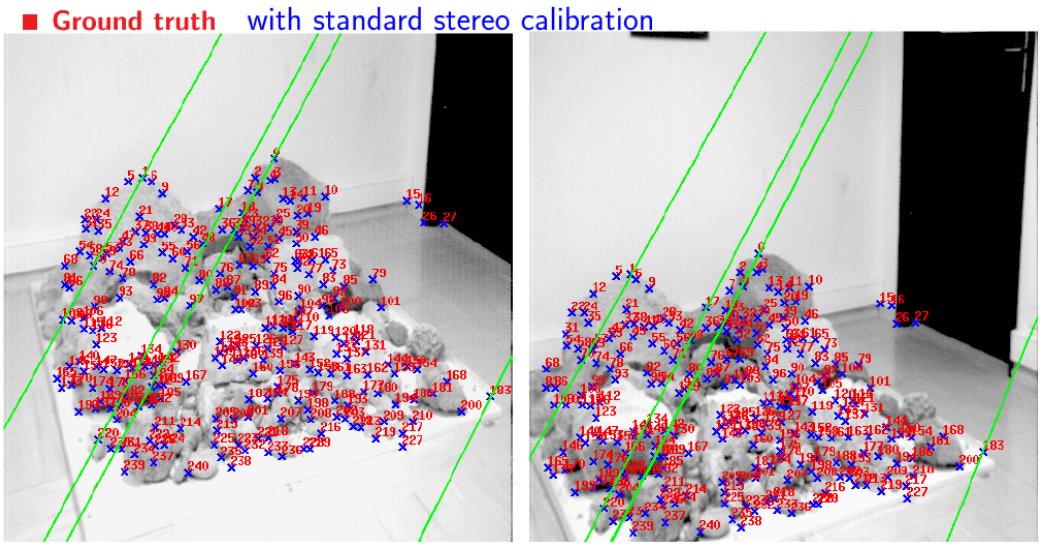
We can replace in this equation the unnormalized image points with the normalized ones, using the inverse of the similarity transforms.
From this expression, we recognize the three inner terms as being equal to the intermediate fundamental matrix that satisfies the Longuet-Higgins

equation for the normalized image points. We can then solve this equality for the final fundamental matrix, to obtain the expression we used in our third step.

Normalized 8-point algorithm

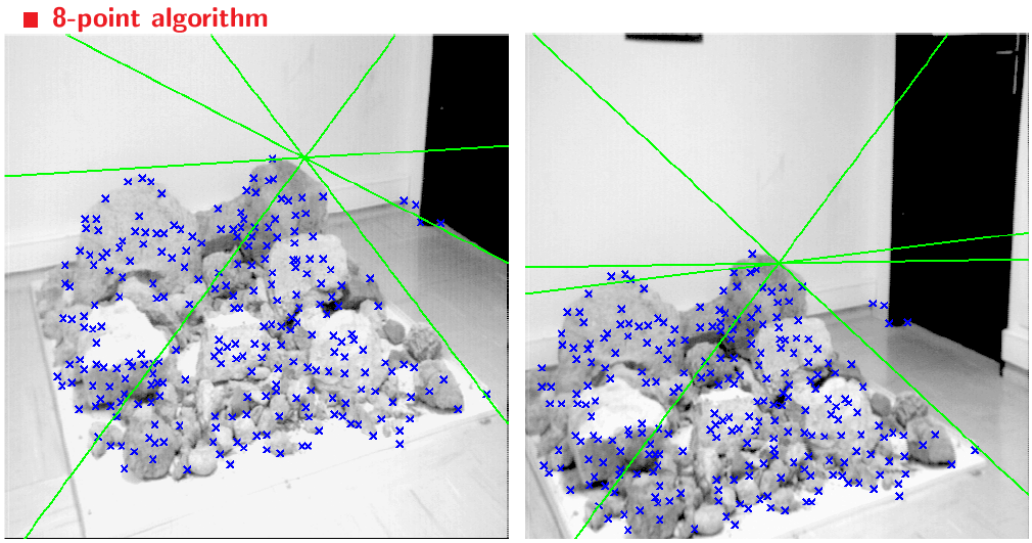
```
[p1, T1] = normalise2dpts(p1);
[p2, T2] = normalise2dpts(p2);

A = [p2(1,:)'.*p1(1,:)'   p2(1,:)'.*p1(2,:)'  p2(1,:)' ...
     p2(2,:)'.*p1(1,:)'   p2(2,:)'.*p1(2,:)'  p2(2,:)' ...
     p1(1,:)'             p1(2,:)'          ones(npts,1) ];

[U,D,V] = svd(A);

F = reshape(V(:,9),3,3)';

[U,D,V] = svd(F);
F = U*diag([D(1,1) D(2,2) 0])*V';


% Denormalise
F = T2'*F*T1;
```

Results (ground truth)



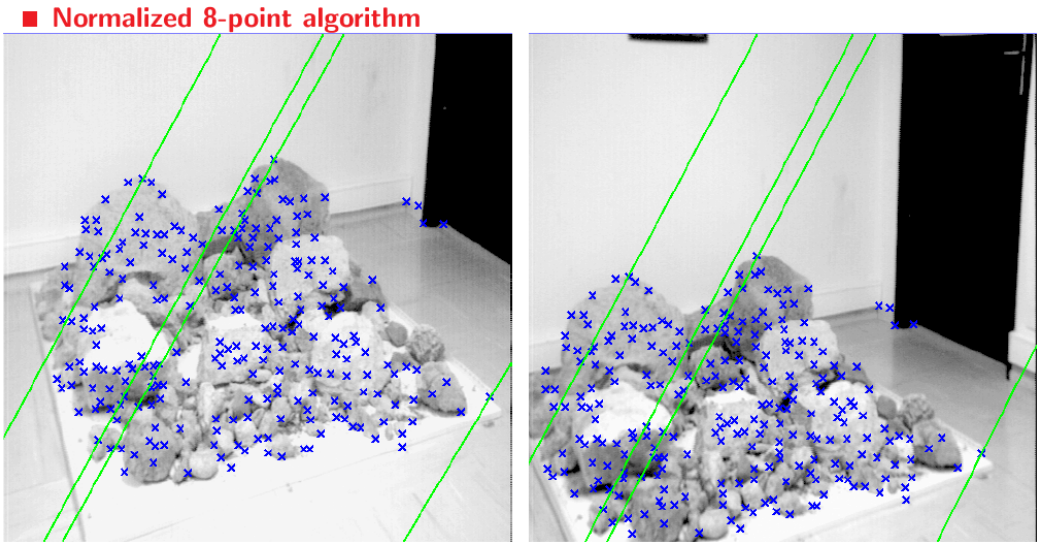
Here is a quick example of the impact normalization can have. We have two images, where we draw groundtruth epipolar lines, computed using standard camera calibration.

Results (8-point algorithm)



If we run the 8-point algorithm without normalization, and use the fundamental matrix we obtain to draw epipolar lines, we see that the results are completely different from the groundtruth. What has happened is that the linear-least-squares solution has overfitted to the large-magnitude entries in our matrix, producing incorrect epipoles near the centers of the images.

Results (normalized 8-point algorithm)



Note: Normalization procedure can benefit other geometry-based vision routines (e.g., DLT for homography estimation, camera calibration)

By contrast, if we run the 8-point algorithm using normalization, and then use the resulting fundamental matrix to draw epipolar lines, the results we get are near-identical to the groundtruth. We see, then, how big of a difference using normalization makes.

We have described the normalization procedure in the context of the 8-point algorithm. But note that normalization can benefit all of the geometry-based vision algorithms we have seen in this and the previous two modules, including the direct linear transform, camera calibration, and so on. All of those algorithms have variants that use normalization and unnormalization steps at the start and end.

Two-view structure from motion

	Structure (scene geometry)	Motion (camera geometry)	Measurements
Pose Estimation	known	estimate	3D to 2D correspondences
Triangulation	estimate	known	2D to 2D coorespondences
Reconstruction	estimate	estimate	2D to 2D coorespondences

Structure from motion

Драконъ, видимый подъ различными углами зрѣнія
По гравюрѣ на мѣди изъ „Oculus artificialis telediopticus" Цана. 1702 года.

Camera calibration & triangulation

- Suppose we know 3D points
  - And have matches between these points and an image
  - How can we compute the camera parameters?

- Suppose we have know camera parameters, each of which observes a point
  - How can we compute the 3D location of that point?

Structure from motion

- SfM solves both of these problems *at once*
- A kind of chicken-and-egg problem
  - (but solvable)

Reconstruction

(2 view structure from motion)

Given a set of matched points

$$\{p_i, p_i'\}$$

Estimate the camera matrices

M, M' ← 'motion' (of the cameras)

Estimate the 3D point

P ← 'structure'

Two-view SfM

1. Compute the Fundamental Matrix **F** from points correspondences
   8-point algorithm

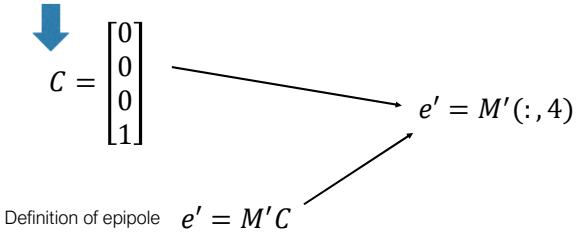
$$p'^T F p = 0$$

The first step in a SFM algorithm is to use the point correspondence and compute the F matrix, using the 8 point algorithm.

Two-view SfM

1. Compute the Fundamental Matrix **F** from points correspondences
   8-point algorithm

2. Compute the camera matrices **M,M'** from the Fundamental matrix
   M = [**I** | **0**] and **M'** = [V(**F**) | **e'**]



From F we kind of reconstruct the camera matrices. We assume the first camera matrix is the identity. We extract the parameters of the second one from F, but we will show below that this extraction is not unique.

First of all from the null space of F we can extract e and e' (up to scale).

We can show that e' is the 4th column of M'.

To see this note that once we select M as [I|0] we assume that is center is at the origin (0,0,0,1);

But the epipole is also the projection of C at the second camera, meaning that e'=M'C. Using the fact that C=[0;0;0;1] we get that e' is the last column of M'.

Extracting the first 3 columns of M from F is more involved and we will see it in the next slides

Camera matrices from essential matrix

$E = [t_\times]R \qquad E = R[dC_\times]$

Null(E) provides e e', and e'=t (last column of camera matrix)

SVD: $\mathbf{E} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top$ Let $\mathbf{W} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

We get FOUR solutions:

$$M' = [R|T]$$

$\mathbf{R}_1 = \mathbf{U}\mathbf{W}\mathbf{V}^\top \quad \mathbf{R}_2 = \mathbf{U}\mathbf{W}^\top\mathbf{V}^\top \qquad \mathbf{T}_1 = U_3 \quad \mathbf{T}_2 = -U_3$

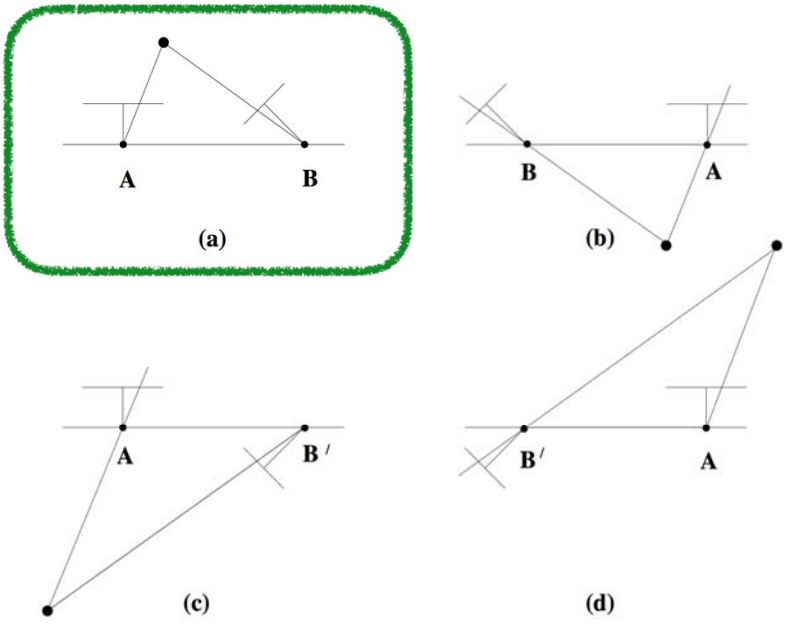
two possible rotations — two possible translations

(See Hartley and Zisserman C.9 for proof)

First of all let us assume we are given the essential matrix E and not F, that is the intrinsic matrix K is known and was already applied on the measured image points p,p'.

We have also derived a formula for E as a function of the rotation and translation of camera 2. From this formula is is easy to see that e' is t, because we want that e'^TE=0, and the cross product of a vector with itself is 0. given the structure of E we get t^T[t_x]=0, hence t^T[t_x]R=0;
Now we also want to extract the rotation matrix from E and there is a formula which tells us how to do that, but it has some ambiguities. The first ambiguity is that when we recover t we only recover it up to scale. In particular we do not recover its direction.
In a similar way there is some ambiguity on the direction of columns in the rotation matrix, and we get 4 possible solutions corresponding to different combinations of these directions.

Find the configuration where the points is in front of both cameras



To understand what the direction ambiguity means geometrically consider this figure. Flipping the direction of t means that camera 2 can be either at the left or at the right of camera 1. Also the sign ambiguity in R means that camera 2 can look forward or look backward. To choose the proper configuration we select one pair of corresponding points in the two cameras, p p'. And we use triangulation to recover its 3D position P. If we selected the wrong sign for R or t, we will get that P is behind one of the cameras and not in front of it.
We simply try the 4 configuration and keep the one which is physically correct.

Remember that since we define the projection of a 3D point as a matrix multiplication p=MP, this algebraic definition applies also to points behind the camera. In practice, in the real world a camera cannot see points behind it, and we use this notion to rule out infeasible solutions.

Camera matrices from fundamental matrix

Compute e, e' as nullspace of F

$$M = [I|0] \quad M' = [V(F)|e']$$

$$V = [e'_\times]F + e'v$$

With v any 1x3 vector

This is a free parameter

V provides some "valid" solution (which is not unique), can not really extract the correct R and K, see below

(See Hartley and Zisserman C.9 for proof)

So if we have the essential matrix we can extract M' uniquely (up to scale). If we have only F and not E there is another formula for extracting the first 3 columns of M' from F.

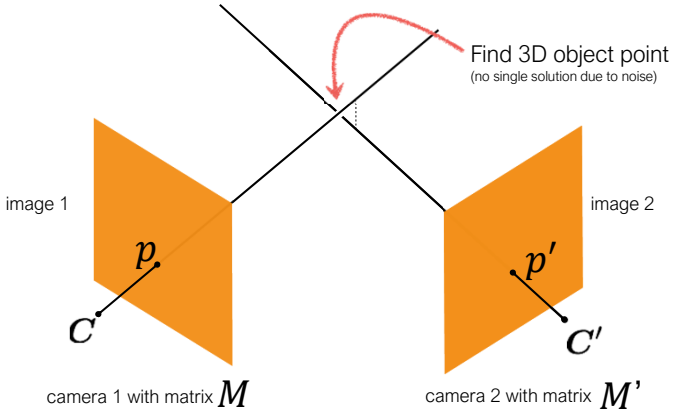
We are not going to prove it in this class (you are welcomed to look up the proof in the Hartley and Zisserman book), but I want you to note that the reconstruction is not unique. There is a 3 parameters vector v that we can choose arbitrarily.

Two-view SfM

1. Compute the Fundamental Matrix **F** from points correspondences
   8-point algorithm
2. Compute the camera matrices **M,M'** from the Fundamental matrix
   M = [**I** | **0**] and **M'** = [V(**F**) | **e'**]
3. For each point correspondence, compute the point **P** in 3D space (triangularization)
   DLT with p = **M** P and p' = **M'** P

Once we estimated the camera matrices we use triangulation to compute the 3D point which explains the corresponding pair of 2D image points p,p'.

Triangulation

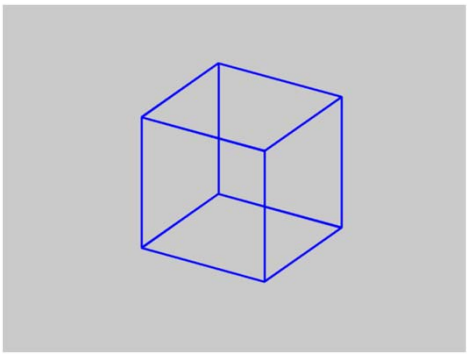


We discussed triangulation in the beginning of lecture 8 and derived the set of linear equations we need to solve to get P.

Ambiguities in structure from motion

Is SfM always uniquely solvable?

- No…



Now you may ask if we have a unique solution to the SFM problem and you may already suspect that the answer is no.
For example in our extraction of camera matrix from F we assumed the first camera is M=[I,0]. This is of course an arbitrary decision and the camera can be anywhere in space.
Also even after we set M, we said that the reconstruction of M' from F had 3 degrees of freedom (we could select and arbitrary 3x1 vector v).

Projective Ambiguity

- Reconstruction is ambiguous by an arbitrary 3D projective transformation without prior knowledge of camera parameters

Structure from motion ambiguity

- If we scale the entire scene by some factor *k* and, at the same time, scale the camera matrices by the factor of 1/*k*, the projections of the scene points in the image remain exactly the same:

$$p = MP = \left(\frac{1}{k}M\right)(kP)$$

$$p' = M'P = \left(\frac{1}{k}M'\right)(kP)$$

It is impossible to recover the absolute scale of the scene!

To see why the reconstruction is not unique note for example we can multiply our scene by some scale factor and also multiply the camera matrices M,M' by the inverse scale, and we would still explain our measurements.

Structure from motion ambiguity

- If we scale the entire scene by some factor *k* and, at the same time, scale the camera matrices by the factor of 1/*k*, the projections of the scene points in the image remain exactly the same

- More generally: if we transform the scene using a transformation **Q** and apply the inverse transformation to the camera matrices, then the images do not change

$$p = MP = (MQ^{-1})(QP)$$
$$p' = M'P = (M'Q^{-1})(QP)$$

Quiz

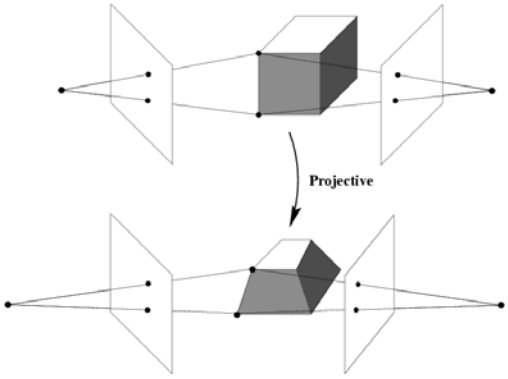
$$p = MP = (MQ^{-1})(QP)$$
$$p' = M'P = (M'Q^{-1})(QP)$$

Suppose we set M=[I|0]. Do we have additional degrees of freedom in the reconstruction?
What Q matrices can we squeeze?

Answer: $Q^{-1} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ v_1 & v_2 & v_3 & v_4 \end{bmatrix}$

$MQ^{-1} = [I|0]$ but yet QP can be quite different than P

Projective ambiguity



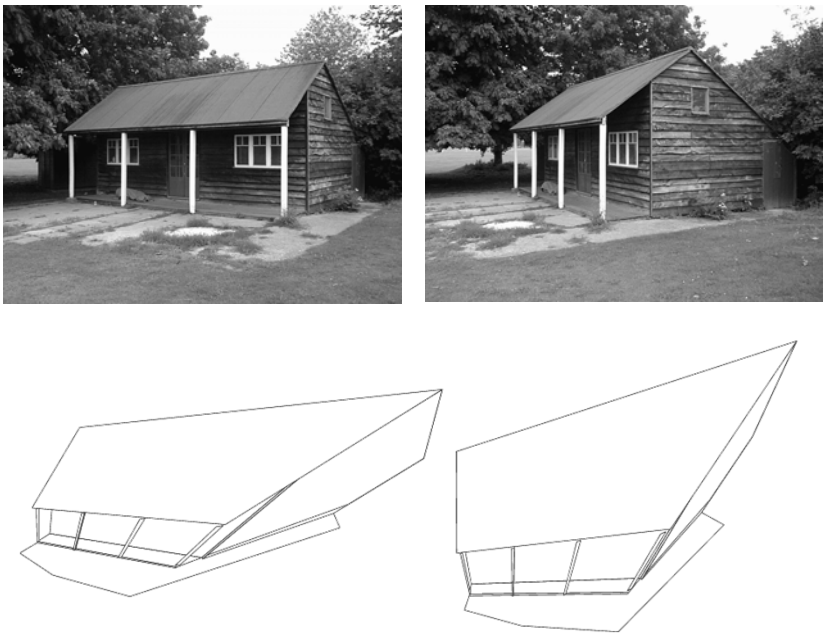
$$p = MP = (MQ^{-1})(QP)$$

The fact that we can apply arbitrary 4x4 transformation to the reconstruction means that we can get wired things. In particular a square cube in the world can be reconstructed as a trapezoid.

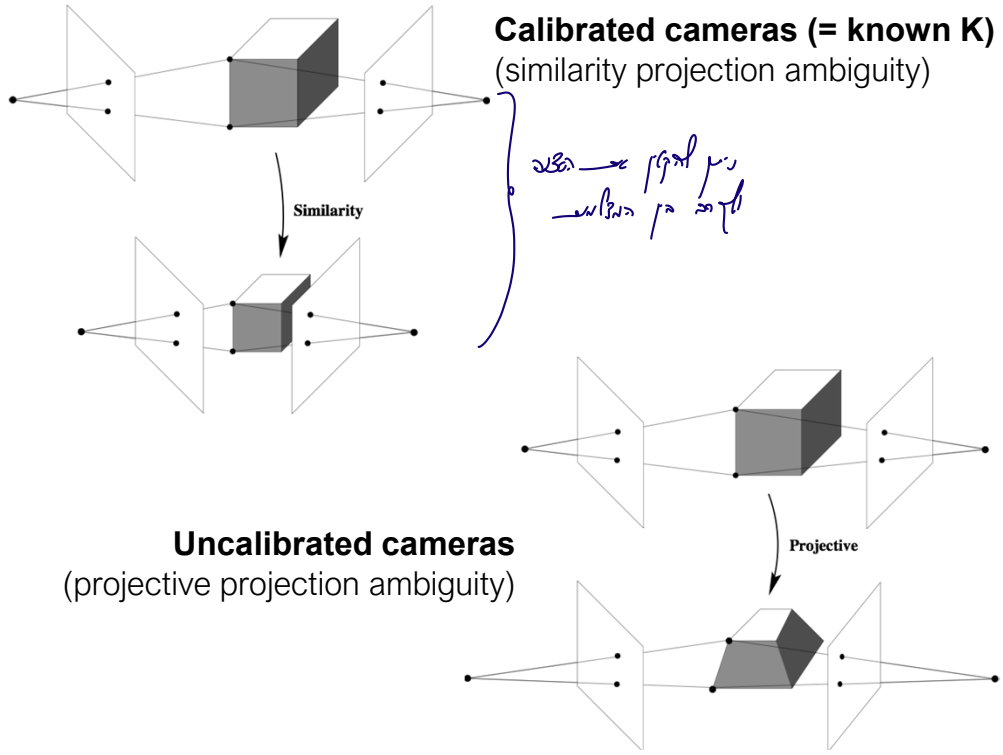
If we can apply any 4x4 transformation on the reconstruction, we have an ambiguity that we call a projective ambiguity.

In such a transformation parallel lines may not be reconstructed as parallel lines and right angles may not be reconstructed as such.

Projective ambiguity



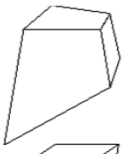
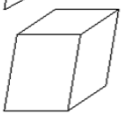
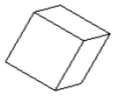
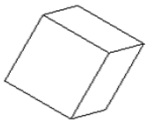
Here are some wired things you can get when you have a projective ambiguity.



In some cases we can get a smaller ambiguity.

For example if we know the intrinsic matrix K we have the additional constraint that the first 3 columns of our M, M' matrices should be of the form KR, where K is known and R is a rotation matrix. This is a strong constraint and we can show (will not show here) that in this case we only have a similarity ambiguity. That is we can rotate and scale the reconstruction, but we cannot distort it more seriously. For example parallel line in the world would be reconstructed as parallel lines, and right angles would be reconstructed as right angles.

Types of ambiguity

Projective 15dof	$\begin{bmatrix} A & t \\ v^T & v \end{bmatrix}$		Preserves intersection and tangency
Affine 12dof	$\begin{bmatrix} A & t \\ 0^T & 1 \end{bmatrix}$		Preserves parallellism, volume ratios
Similarity 7dof	$\begin{bmatrix} sR & t \\ 0^T & 1 \end{bmatrix}$		Preserves angles, ratios of length
Euclidean 6dof	$\begin{bmatrix} R & t \\ 0^T & 1 \end{bmatrix}$		Preserves angles, lengths

- With no constraints on the camera calibration matrix or on the scene, we get a *projective* reconstruction
- Need additional information to *upgrade* the reconstruction to affine, similarity, or Euclidean

Slide: S. Lazebnik

Here is a classification of the types of ambiguities we can have. This is very similar to the classification of 2D transformations we have seen in the homography lecture and please review again the definitions there as they were more detailed than what we have here.

If we know nothing the matrix Q we can squeeze is a general 4x4 matrix and we can transform a cube into any trapezoid.

In some cases we have additional information and when we use additional constraints we have fewer degrees of freedom in Q.

For example, we can have additional constraints which imply that the last row of Q should be (0,0,0,1).

One way to obtain such information that you will see in the home assignment, is when you can identify a pair of lines that should be parallel and enforce the fact that they are reconstructed as such.

In this case the ambiguity is smaller and a cube can only be transformed into a parallelogram. In particular in this transformation parallel lines remain parallel.

As we said if we know the intrinsic matrix one can show (we won't prove) that the ambiguity on Q reduces to a similarity ambiguity where we can rotate and scale a

cube but right angles and parallel lines in the world would be reconstructed as such.

Finally we say we have Euclidian ambiguity if Q is only a rotation and transformation. This kind of ambiguity preserves lengths.

Multi-view projective structure from motion

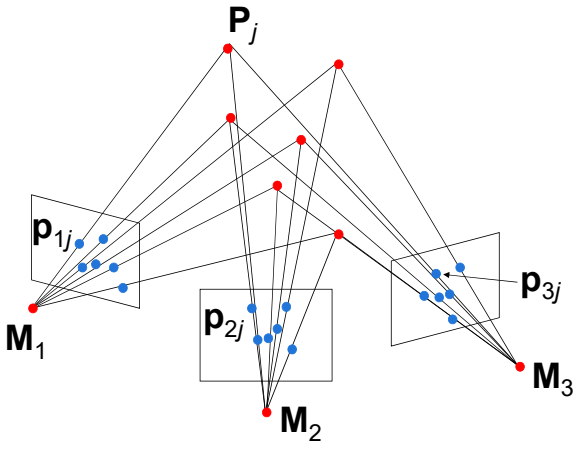
So far we have seen how to do reconstruction from 2 cameras looking at the scene from 2 different viewpoints.
Clearly if we have more than 2 cameras looking at the same scene we can get better results.

Projective structure from motion

- Given: *m* images of *n* fixed 3D points

$$\text{Scale}\ z_{ij}\mathbf{p}_{ij} = \boldsymbol{M}_i \mathbf{P}_j, \quad i = 1, \ldots, m, \quad j = 1, \ldots, n$$

- Problem: estimate *m* projection matrices $\mathbf{M}_i$ and *n* 3D points $\mathbf{P}_j$ from the *mn* correspondences $\mathbf{p}_{ij}$



In the general setting we have n 3D points and we view them from m cameras.

We are given n*m 2D projections of these points in the cameras and want to estimate the n 3D point and the m camera matrices.

Projective structure from motion

- Given: *m* images of *n* fixed 3D points

$$z_{ij}\,\mathbf{p}_{ij} = \mathbf{M}_i\,\mathbf{P}_j, \quad i = 1,\ldots,m, \quad j = 1,\ldots,n$$

- Problem: estimate *m* projection matrices $\mathbf{M}_i$ and *n* 3D points $\mathbf{P}_j$ from the *mn* correspondences $\mathbf{p}_{ij}$
- With no calibration info, cameras and points can only be recovered up to a 4x4 projective transformation **Q**:

$$\mathbf{P} \rightarrow \mathbf{QP},\ \mathbf{M} \rightarrow \mathbf{MQ}^{-1}$$

- We can solve for structure and motion when

$$2mn >= 11m + 3n - 15$$

- For two cameras, at least 7 points are needed

Q: To do FSM from 2 cameras we first need to estimate F, and for which we have derived the 8 point algorithm? So can we do SFM from 7 points rather than 8?

A: The 8 point algorithm neglects another non linear constraint stating that F should be a rank 2 matrix. When this constraint is used we can solve from 7 points.

As said earlier with no additional information the reconstruction is defined up to a 4x4 transformation Q.

We want to understand when we have enough constraints to solve this problem.

Each 2D point gives us 2 constrains since we need to ensure that p_ij=M_iP_j up to scale.

So from n*m 2D points we get 2nm constraints.

Now lets count the number of unknowns. In each camera we have 11 unknowns (it is a 3x4 matrix defined up to scale).

In each 3D point we have 3 unknowns. So in total we have 11m+3n unknowns.

From this we subtract the 15 degrees of freedom in Q since this is an ambiguity we will always have (even given a lot of matching 2D points).

So in total we get that to have a unique solution we need 2nm>=1m+3n-15. This tells us that there is a minimal number of corresponding 2D points that we need to detect to solve the problem.

In particular we get that for 2 cameras (m=2) we need n>=7 corresponding points.

Now you may ask: we have derived the solution for the SFM problem from 2

cameras, and we said that we need to start by finding F, and for that we have derived a linear system and we need at least 8 points to solve this system uniquely.
The answer is that if we think of F as a 3x3 matrix defined up to scale, it indeed has 8 degrees of freedom.
But there is an additional constraint that F should be a rank 2 matrix, and this brings the degrees of freedom in F down from 8 to 7.
The 8 point algorithm does not use this additional rank constraint because it is a non linear constraint. But it can be incorporated into a non linear optimization strategy.

Projective SFM: Two-camera case

- Compute fundamental matrix $\mathbf{F}$ between the two views
- First camera matrix: $[\mathbf{I}|\mathbf{0}]$
- Second camera matrix: $[\mathbf{A}|\mathbf{b}]$
- Then $\mathbf{b}$ is the epipole ($\mathbf{F}^T\mathbf{b} = 0$), $\mathbf{A} = V(\mathbf{F})$ (see formula in a previous slide)

To do SFM from multiple views we typically use incremental strategies. We first do SFM from 2 cameras and get some estimate of 3D. Then we add another camera and refine and so on.
Here is a reminder of how we do SFM using 2 cameras.

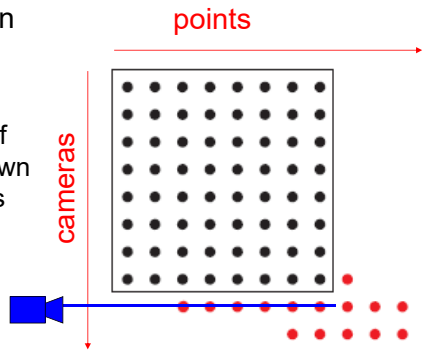
Sequential structure from motion

•Initialize motion from two images using fundamental matrix

•Initialize structure by triangulation

•For each additional view:

- Determine projection matrix of new camera using all the known 3D points that are visible in its image – *calibration*



We use the first 2 cameras to get an initialization for the 3D structure (once we solved for camera matrices we can use triangulation).
Then we can use the 3D points to find the camera projection matrix explaining the 3D points and the 2D measurements marked in the camera we want to add.

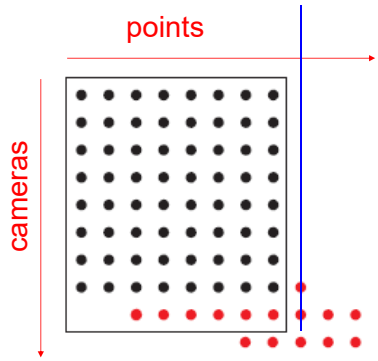
Sequential structure from motion

•Initialize motion from two images using fundamental matrix

•Initialize structure by triangulation

•For each additional view:

- Determine projection matrix of new camera using all the known 3D points that are visible in its image – *calibration*
- Refine and extend structure: compute new 3D points, re-optimize existing points that are also seen by this camera – *triangulation*



Once we have another camera we can do triangulation again using the new camera and refine the 3D reconstruction.

Note that not all points are visible by all cameras. Either because objects in the scene occlude each other, or because when we moved the camera some content gets out of the frame, or because we just have noise and the SIFT detector did not find all correspondences.
This is fine and we reconstruct each 3D point from the cameras in which we have the projection.

The diagram here shows which points are visible by which camera.

In particular when we reconstruct camera number j we may have new points that we did not reconstruct in previous frames and we will need to do triangulation to reconstruct them now.
Here we mark such a new point with a blue line.

Sequential structure from motion

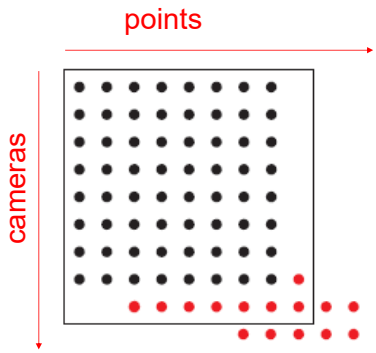
•Initialize motion from two images using fundamental matrix

•Initialize structure by triangulation

•For each additional view:

- Determine projection matrix of new camera using all the known 3D points that are visible in its image – *calibration*
- Refine and extend structure: compute new 3D points, re-optimize existing points that are also seen by this camera – *triangulation*

•Refine structure and motion: bundle adjustment

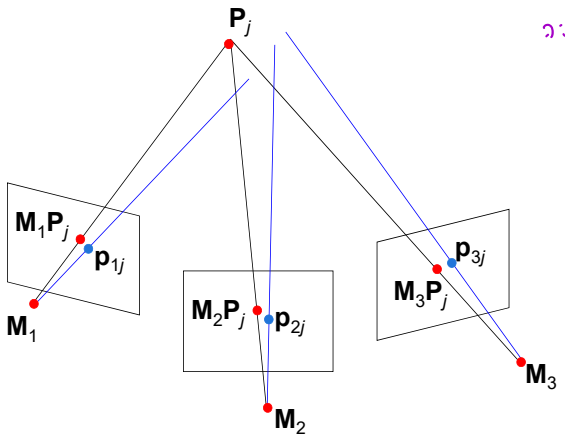


So far we have only used linear reconstruction algorithm. But in fact the linear reconstruction algorithms do not minimize the right error metric. Therefore once we have the linear reconstruction we use it to initialize a non linear algorithm that can minimize a better error metric and get a more accurate reconstruction. Such a non linear refinement is called bundle adjustment.

Bundle adjustment

- Non-linear method for refining structure and motion
- Minimizing reprojection error

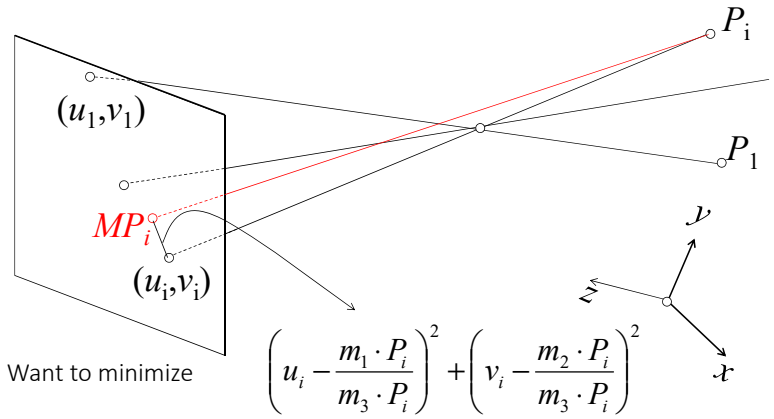
$$E(M,P) = \sum_{i=1}^{m}\sum_{j=1}^{n} D\left(p_{ij}, M_i P_j\right)^2$$



The error we want to minimize is the distance between the projection MiPj after it was converted to heterogeneous coordinates, and the marked 2D points pij, again in heterogeneous coordinates.

This is not exactly what we minimize in our linear estimation algorithms.

Reminder: Minimizing reprojection error



Linear methods minimize $(u_i(m_3P_i)-m_1P_i)^2+(v_i(m_3P_i)-m_2P_i)^2$

A reminder on what we want to minimize and what we minimize in practice.
To minimize the actual reprojection error problem we need a non linear optimization

Review: Structure from motion

- Ambiguity
- Dealing with missing data
  - Incremental structure from motion
- Projective structure from motion
  - Bundle adjustment

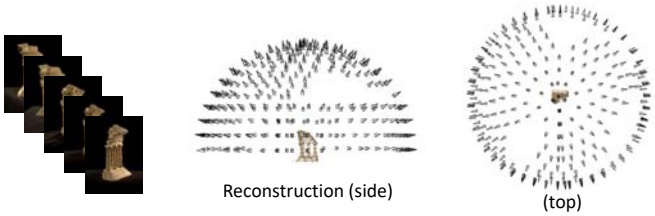
This summarize our discussion about structure from motion

	Structure (scene geometry)	Motion (camera geometry)	Measurements
Pose Estimation	known	estimate	3D to 2D correspondences
Triangulation	estimate	known	2D to 2D coorespondences
Reconstruction	estimate	estimate	2D to 2D coorespondences

Large-scale structure from motion

In the next slides we will review a very nice project that solved a structure from motion problem in a very large scale.

Structure from motion



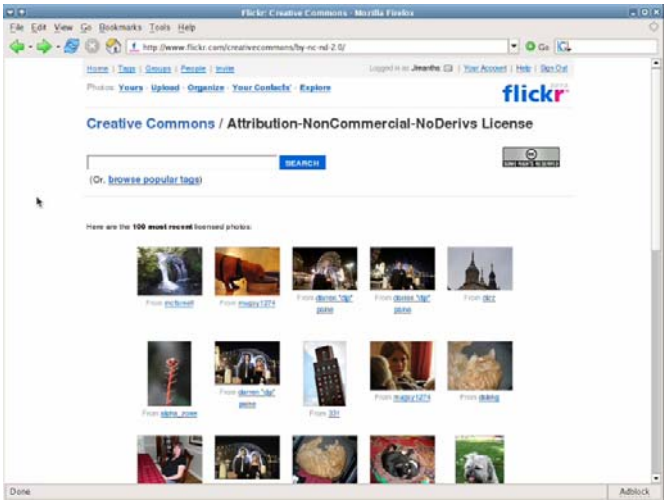
- Input: images with points in correspondence
  $p_{i,j} = (x_{i,j}, y_{i,j})$
- Output
  - structure: 3D location $\mathbf{P}_i$ for each point p_i
  - motion: camera parameters $\mathbf{R}_j$, $\mathbf{t}_j$ possibly $\mathbf{K}_j$
- Objective function: minimize *reprojection error*

15,464

37,383

76,389

Standard way to view photos



Advances in digital camera technology and the popularity of photo-sharing sites such as Flickr have resulted in the availability of vast numbers of photos. Doing a Flickr search for a famous place like the Trevi Fountain, returns thousands of photos. Unfortunately, common image browsing tools don't give a sense of a rich relationships between all of the photos.

The goal of Photo Tourism is to use location information to build a better visualization and photo exploration tool for collections of photos of the same scene.

[If I do a search for Trevi and Rome in Flickr, you'll see a typical way of browsing such collections – looking at page after page of thumbnails. If I have a specific kind of photo of the Trevi Fountain in mind, it can be hard to find it this way, and the relationships between all the photos isn't clear.

The goal of the Photo Tourism project is to use location information to build a better visualization and photo exploration tool for collections of photos of

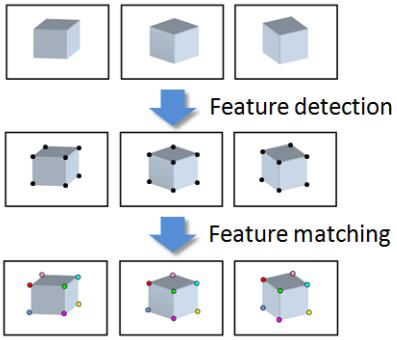
the same scene.]

[Here's a sample search for "Trevi Fountain" on Flickr – results are returned on pages and pages of thumbnails. Finding a photo of a specific part of the fountain amounts to scrolling through the thumbnails, and it's not clear how all the photos are related to each other.
The goal of the Photo Tourism project is to use location information to build a better visualization and photo exploration tool for collections of photos of the same scene.]

Phot Tourism

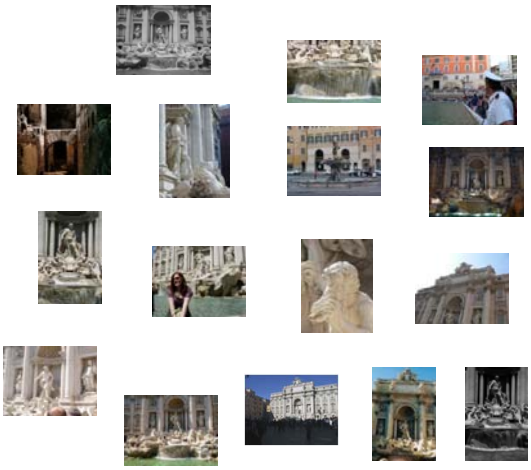
Here's how the same set of photos appear in our photo explorer. Our system takes the set of photos and automatically determines the relative positions and orientations from which each photo was taken. We can then load the photos into our immersive 3D browser where the user can visualize and explore the photos using spatial relationships.

Input: Point correspondences



Feature detection

Detect features using SIFT [Lowe, IJCV 2004]

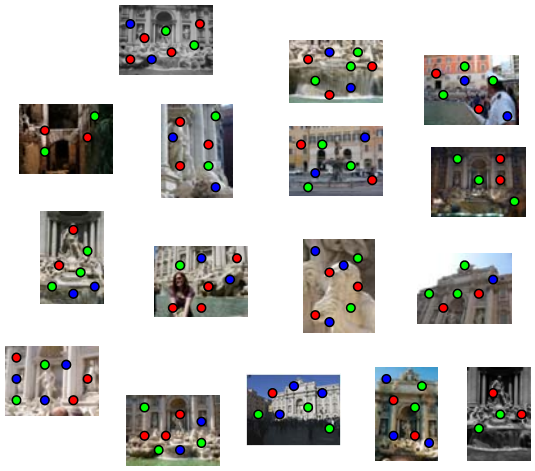


So, we begin by detecting SIFT features in each photo.

[We detect SIFT features in each photo, resulting in a set of feature locations and 128-byte feature descriptors…]

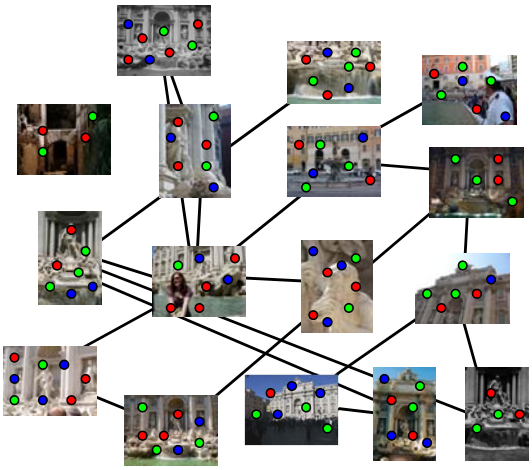
Feature description

Describe features using SIFT [Lowe, IJCV 2004]



Feature matching

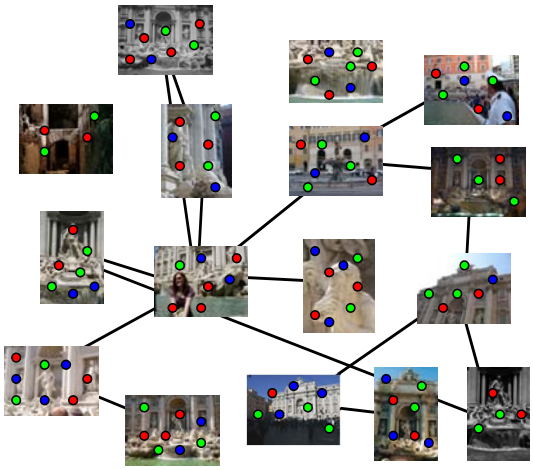
Match features between each pair of images



Next, we match features across each pair of photos using approximate nearest neighbor matching.

Feature matching

Refine matching using RANSAC to estimate fundamental matrix between each pair



The matches are refined by using RANSAC, which is a robust model-fitting technique, to estimate a fundamental matrix between each pair of matching images and keeping only matches consistent with that fundamental matrix. We then link connected components of pairwise feature matches together to form correspondences across multiple images.

Once we have correspondences, we run structure from motion to recover the camera and scene geometry.

Correspondence estimation

- Link up pairwise matches to form connected components of matches across several images

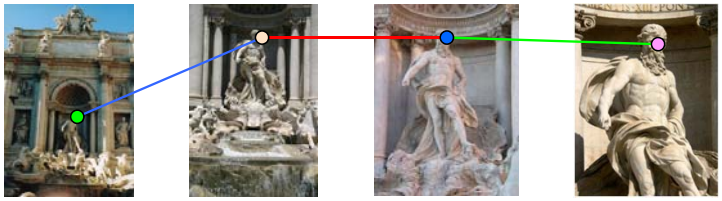
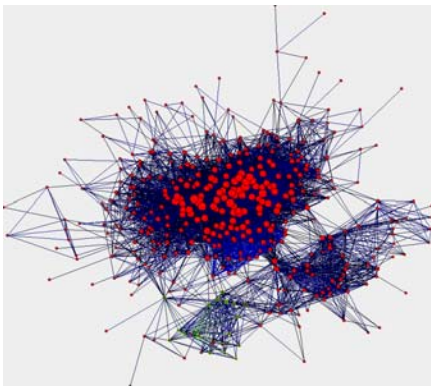


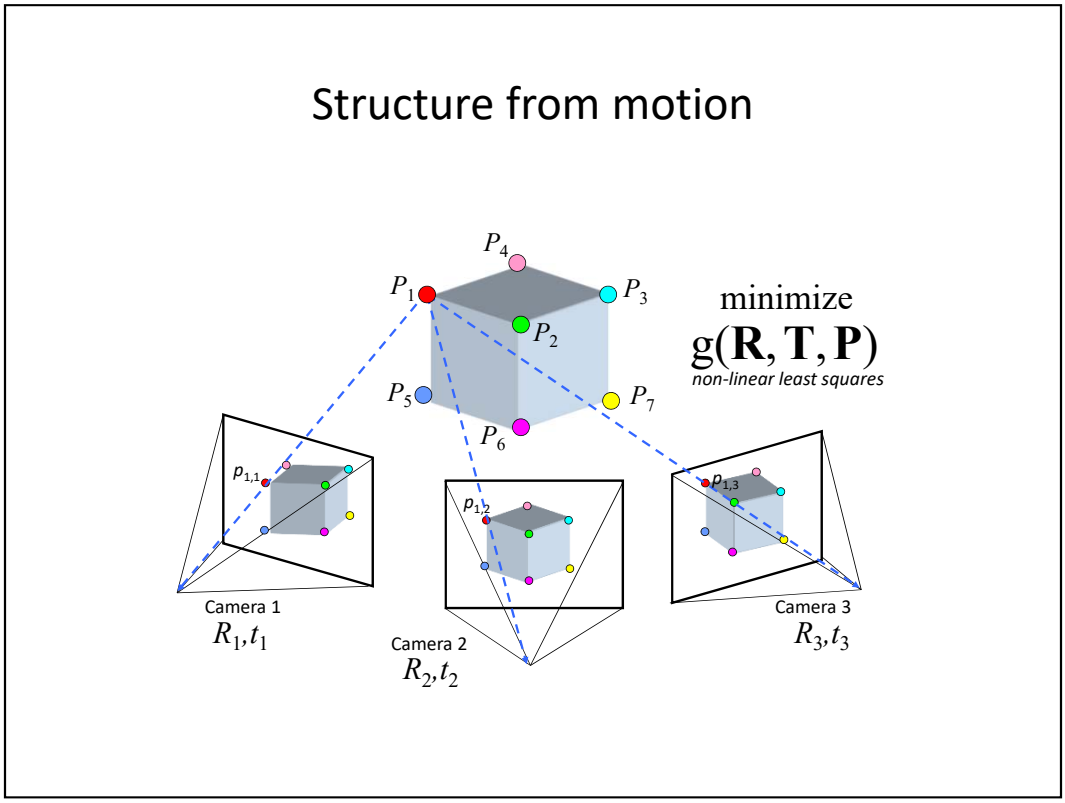
Image 1 Image 2 Image 3 Image 4

The next step of our reconstruction algorithm is to link up matches between pairs of images into correspondences between multiple images, by finding connected components of matches. Suppose among these images, these features are matched only between consecutive pairs of images, perhaps because the scales of the images are so different. During correspondence estimation we link these matches and consider them to be 2D projections of a single 3D point in the scene.

Image connectivity graph



(graph layout produced using the Graphviz toolkit: http://www.graphviz.org/)



Structure from motion solves the following problem:

Given a set of images of a static scene with 2D points in correspondence, shown here as color-coded points, find…

a set of 3D points P and
a rotation R and position t of the cameras that explain the observed correspondences. In other words, when we project a point into any of the cameras, the reprojection error between the projected and observed 2D points is low.
This problem can be formulated as an optimization problem where we want to find the rotations R, positions t, and 3D point locations P that minimize sum of squared reprojection errors f. This is a non-linear least squares problem and can be solved with algorithms such as Levenberg-Marquart. However, because the problem is non-linear, it can be susceptible to local minima. Therefore, it's important to initialize the parameters of the system carefully. In addition, we need to be able to deal with erroneous

correspondences.

Global structure from motion

- Minimize sum of squared reprojection errors:

$$E(P,R,T) = \sum_{i=1}^{m}\sum_{j=1}^{n} \underbrace{w_{ij}}_{\substack{\text{indicator variable:}\\ \text{is point } i \text{ visible in image } j \text{ ?}}} \left\| \underbrace{Proj([R_j|t_j]P_j)}_{\substack{\text{predicted}\\ \text{image location (heterogenous)}}} - \underbrace{\begin{bmatrix} x_{ij} \\ y_{ij} \end{bmatrix}}_{\substack{\text{observed}\\ \text{image location}}} \right\|^2$$

- Minimizing this function is called *bundle adjustment*
  - Optimized using non-linear least squares, e.g. Levenberg-Marquardt

Problem size

- What are the variables?
- How many variables per camera?
- How many variables per point?

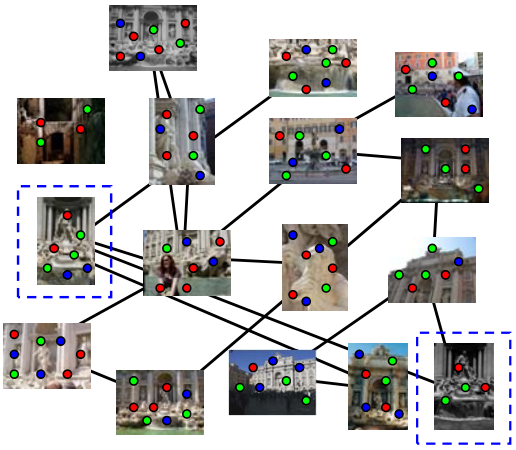
- Trevi Fountain collection

  466 input photos

  \+ > 100,000 3D points

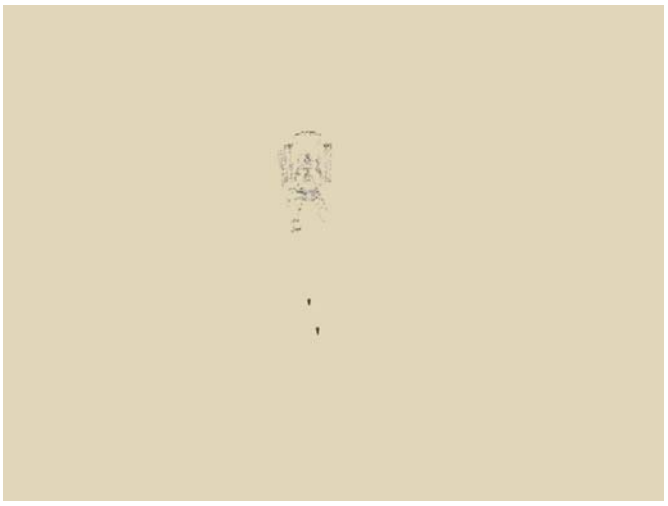
  = very large optimization problem

Initialization: Incremental structure from motion



To help get good initializations for all of the parameters of the system, we reconstruct the scene incrementally, starting from two photographs and the points they observe.

Incremental structure from motion



We then add several photos at a time to the reconstruction, refine the model, and repeat until no more photos match any points in the scene.

Final reconstruction

Even larger scale SfM

City-scale structure from motion

- “Building Rome in a day”

http://grail.cs.washington.edu/projects/rome/

Discrete vs. continues depth

- Depth reconstruction requires a pair of matching points p, p’ as input. This matching can only be applied at image regions with considerable texture (e.g. corners) rather than uniform regions

- Next lecture discuss the concept of stereo matching, attempting to interpolate depth values in texture-less regions.

Robustness of depth reconstruction

- Robust reconstruction relies on sufficient baseline between cameras. At the extreme case if two cameras have a join center the two rays we are trying to triangulate are the same ray and their intersection is not unique

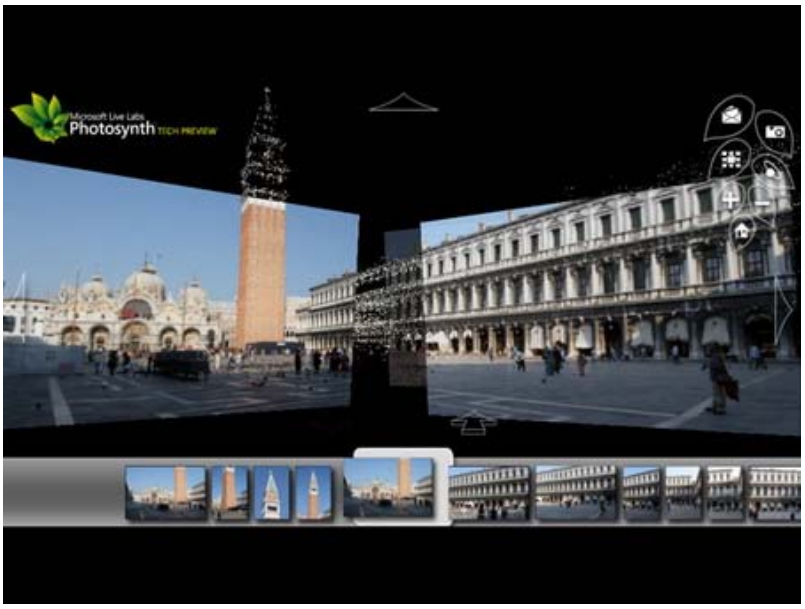
(We also said that in this case we can map between the two images using a single homography, another evident that no 3D parallax is present)

SfM applications

- 3D modeling
- Surveying
- Robot navigation and mapmaking
- Visual effects (“Match moving”)
  - https://www.youtube.com/watch?v=RdYWp70P_kY


PEPSI MAX
PRESENTS

Applications – Photosynth



Applications – Hyperlapse

https://www.youtube.com/watch?v=SOpwHaQnRSY

Summary: 3D geometric vision

- Single-view geometry
  - The pinhole camera model
    - Variation: orthographic projection
  - The perspective projection matrix
  - Intrinsic parameters
  - Extrinsic parameters
  - Calibration
- Multiple-view geometry
  - Triangulation
  - The epipolar constraint
    - Essential matrix and fundamental matrix
  - Stereo
    - Binocular, multi-view
  - Structure from motion
    - Reconstruction ambiguity
    - Projective SFM

References

Basic reading:

- Szeliski textbook, Chapter 7.
- Hartley and Zisserman, Chapter 9,18.